



SAPIENZA
UNIVERSITÀ DI ROMA

Sviluppo di un Modello di Classificazione e Segmentazione Automatica del Medulloblastoma su Immagini MRI

Facoltà di Ingegneria dell'informazione, Informatica e Statistica
Corso di Laurea in Informatica

Candidato

Enrico Chiarello

Matricola 2000457

Relatore

Prof. Danilo Avola

Correlatore

Prof. Daniele Pannone

Anno Accademico 2023/2024

Sviluppo di un Modello di Classificazione e Segmentazione Automatica del Medulloblastoma su Immagini MRI

Tesi di Laurea. Sapienza – Università di Roma

© 2024 Enrico Chiarello. Tutti i diritti riservati

Questa tesi è stata composta con L^AT_EX e la classe Sapthesis.

Email dell'autore: enrico02chiarello@gmail.com

Indice

1	Introduzione	1
1.1	Ambito e scopo dell'applicazione	1
1.2	Stato dell'arte	2
1.2.1	Confronto con lo Stato dell'Arte	3
1.3	Il medulloblastoma	4
1.3.1	Storia del disturbo	4
1.3.2	Diagnosi differenziale	4
1.3.3	Diffusione e imaging	4
1.3.4	Caratteristiche radiologiche	5
1.3.5	Aspetti istologici	5
2	Metodo	7
2.1	Introduzione	7
2.2	Machine learning	7
2.3	Deep learning	8
2.3.1	Applicare deep learning ad immagini mediche	9
2.4	Librerie per l'Elaborazione e il Caricamento dei Dati	12
2.4.1	Sklearn	12
2.4.2	Kares e TensorFlow	12
2.4.3	Matplotlib	12
2.4.4	Seaborn	13
2.4.5	Pandas	13
2.4.6	NumPy	13
2.5	Reti Neurali	14
2.5.1	Neurone Biologico	14
2.5.2	Neurone Artificiale	14
2.6	Reti neurali Artificiali	16
2.6.1	Processo di Addestramento	17
2.7	Backpropagation	18
2.8	Gradient Descent	19
2.9	Modelli Neurali	20
2.9.1	Reti Convoluzionali	20
2.9.1.1	Architettura di una CNN	21
2.9.1.2	Funzionamento delle CNN	22
2.9.1.3	Applicazioni delle CNN	22
2.9.1.4	Vantaggi delle CNN	23
2.9.1.5	Limitazioni delle CNN	23

3	Esperimenti	25
3.1	Dataset	25
3.1.1	Strumento per la creazione di maschere	26
3.2	Classificazione	26
3.2.1	Data Augmentation del dataset	26
3.2.2	Pre-processamento	27
3.2.2.1	Normalizzazione	27
3.2.2.2	Sample-wise Centering e Sample-wise Standardization	27
3.2.3	Caricamento dei dati	27
3.2.4	Modello pre-addestrato	27
3.2.5	Fine-tuning e Architettura del Modello	29
3.2.6	Compilazione del modello	30
3.2.6.1	Ottimizzatore Adam	30
3.2.7	Addestramento del modello	30
3.2.8	Previsioni del modello sul validation set	30
3.3	Segmentazione	31
3.3.1	Data Augmentation e Pre-processamento	31
3.3.2	Caricamento dei dati	31
3.3.3	U-Net: Architettura e Vantaggi	31
3.3.4	Training e valutazione finale del modello	33
4	Conclusioni	35
4.1	Risultati della Classificazione	35
4.1.1	Classificazione con aumento del numero di epoche	38
4.2	Risultati della Segmentazione	39
4.3	Epilogo	41
	Bibliografia	43

Capitolo 1

Introduzione

1.1 Ambito e scopo dell'applicazione

Tra le malattie più gravi e oggetto di continua ricerca scientifica troviamo i tumori, specialmente quelli al cervello, che sono particolarmente pericolosi. I tumori cerebrali si distinguono in base a diversi criteri, quali l'origine (primitiva o metastatica), la localizzazione (emisferi cerebrali, cervelletto, tronco encefalico, ecc.), la velocità di crescita (tumori a crescita rapida o lenta) e altre caratteristiche biologiche. Nel campo della bioinformatica, l'analisi istologica microscopica è oggetto di studio per supportare i professionisti nel riconoscimento dei tumori attraverso immagini istologiche digitali.

La diagnosi dei tumori cerebrali avviene utilizzando vari strumenti, tra cui, come primo passo, la risonanza magnetica (MRI). La MRI fornisce immagini dettagliate del cervello e del cranio, permettendo di individuare anomalie strutturali. Esistono diversi tipi di risonanza magnetica, ognuno con applicazioni specifiche in ambito diagnostico. Tra i principali tipi di MRI utilizzati in ambito medico troviamo:

- **T1-ponderata**: acquisisce immagini utilizzando una sequenza con tempo di rilassamento longitudinale, mostrando un contrasto elevato tra il tessuto adiposo e i tessuti molli, con una buona risoluzione spaziale.

- **T2-ponderata**: acquisisce immagini utilizzando una sequenza con tempo di rilassamento trasversale, utilizzata per evidenziare la presenza di liquidi, come edemi o infiammazioni, ed è particolarmente utile per individuare tumori e lesioni cerebrali.

- **T1 con contrasto (T1C+)**: acquisisce immagini utilizzando una sequenza con tempo di rilassamento longitudinale e prevede anche l'iniezione di un mezzo di contrasto, come il gadolinio, per migliorare la visibilità di alcune aree del cervello, specialmente quelle affette da tumori che alterano la barriera emato-encefalica.

Nell'ambito della bioinformatica, diagnosticare un tumore significa eseguire una classificazione e/o una segmentazione delle immagini mediche. Le reti neurali, in particolare le reti neurali convoluzionali (CNN), svolgono un ruolo cruciale in questo processo, analizzando grandi dataset di immagini per predire il tipo di tumore e identificare la sua localizzazione attraverso tecniche di segmentazione. È dunque fondamentale ottenere una classificazione accurata come primo passo, consentendo così una corretta individuazione della massa tumorale e, successivamente, la sua eventuale rimozione chirurgica.

L'applicazione che verrà presentata si concentra inizialmente sulla classificazione delle immagini MRI, distinguendo tra quelle in cui sono presenti medulloblastomi e quelle prive di tumori. Una volta completata la fase di classificazione, si procede

alla segmentazione del tumore, identificando con precisione la regione interessata dalla massa tumorale.

1.2 Stato dell'arte

Attualmente, lo stato dell'arte nel campo della classificazione dei tumori cerebrali ha raggiunto notevoli risultati grazie a tecniche avanzate di intelligenza artificiale, machine learning e deep learning. Tuttavia, la segmentazione automatica dei tumori presenta ancora sfide significative, in particolare in termini di accuratezza e generalizzazione dei modelli.

Alcuni ricercatori dedicano i propri studi allo sviluppo di soluzioni per la segmentazione e piattaforme come BraTS (Brain Tumor Segmentation Challenge) giocano un ruolo centrale nel promuovere la collaborazione della comunità scientifica e nel stimolare il progresso nel settore. Queste competizioni forniscono dataset complessi e reali, permettendo ai partecipanti di testare e migliorare i propri algoritmi.

Uno studio recente ha raccolto dati MRI da 217 pazienti con medulloblastoma, sviluppando un modello di segmentazione automatica basato su una rete 3D-Unet. Questo modello ha dimostrato un'alta precisione nella delineazione del tumore, ottenendo un coefficiente di Dice medio di 0.80 per il training, di 0.82 per il validation e di 0.78 per il test, dimostrando efficacia nel delineare i tumori riducendo i tempi necessari per l'annotazione manuale. Parallelamente, è stato costruito un modello per la predizione della sopravvivenza combinando caratteristiche cliniche e radiomiche, migliorando la predittività rispetto ai singoli fattori clinici.

In ambito medico, per essere utilizzato in pratica clinica, un modello di segmentazione deve raggiungere risultati eccellenti. Generalmente, è richiesto un coefficiente Dice medio superiore a 0.9, dato che un'elevata precisione è fondamentale per garantire l'affidabilità dei risultati, specialmente in applicazioni critiche come la segmentazione di tumori cerebrali.

Molti studi scientifici disponibili online riguardano la classificazione e la segmentazione dei tumori cerebrali, concentrandosi prevalentemente su gliomi, meningiomi e tumori della ghiandola pituitaria. Il numero di articoli che trattano la segmentazione del medulloblastoma, invece, non è elevato ed il motivo principale è che a causa della natura pediatrica e della rarità del medulloblastoma, i pochi dataset disponibili spesso non coprono le diverse caratteristiche radiologiche e istologiche del tumore, come accade ad esempio per i gliomi o altri tumori cerebrali più comuni. Questi fattori rendono la segmentazione automatica del medulloblastoma un campo ancora in fase di esplorazione, con un notevole margine di miglioramento per quanto riguarda la qualità e la validità dei modelli sviluppati.

Questo inoltre ha reso più complessa la ricerca di dataset pubblicamente disponibili di dimensioni sufficienti per sviluppare modelli robusti. La mancanza di dataset pubblici sui medulloblastomi rappresenta una sfida significativa, che potrebbe essere superata attraverso collaborazioni dirette con strutture ospedaliere per ottenere dati più specifici e rilevanti.

Nonostante ciò, è stato possibile rimediare un piccolo dataset di circa 130 immagini di medulloblastoma su Kaggle, una piattaforma online che ospita competizioni di data science e machine learning, fornendo accesso a vari dataset per la ricerca

e lo sviluppo di modelli. Per quanto riguarda il modello di segmentazione, invece, si è proceduto con la creazione manuale delle maschere, che sono immagini binarie utilizzate per identificare e delimitare le regioni di interesse all'interno delle immagini originali. Le maschere evidenziano le aree tumorali, consentendo al modello di apprendere a distinguere tra tessuti sani e patologici.

Importante è stata la collaborazione con il dott. Lorenzo Chiarello, per la segnalazione delle zone anomale delle MRI, fornendo un prezioso supporto nella realizzazione di queste maschere.

1.2.1 Confronto con lo Stato dell'Arte

Il dataset utilizzato (la cui composizione verrà descritta più avanti) proviene da Kaggle, una piattaforma che offre dataset e propone sfide per lo sviluppo di reti neurali. Il dataset è stato costruito unendo due raccolte di dati originariamente composte rispettivamente da 14 e 44 classi, dalle quali è stata selezionata esclusivamente la classe del medulloblastoma. Alcuni sviluppatori hanno scelto di implementare lo stesso task sull'intero dataset, includendo quindi tutte le classi. Tra i vari progetti, ecco i risultati di alcuni modelli di classificazione:

Autore	Train	Validation	Test
Arturo Bandini Jr.	-	-	99%
Matteo Bellitti	-	-	99%
Fathy Sahlool	99%	96%	97%
Shrijeeth S	-	98%	97%
Eyad Aiman	-	98%	97%
Abdallah Wagih Ibrahim	99%	96%	96%
Enrico Chiarello	90%	94%	96%
Diego Buitrago Calcerrada	-	-	96%
Waseem Nagah Henes	-	-	95%
Gerry	-	-	95%
Ahmed Raafat Mohamed	99%	95%	95%
Ahmed Ra'afat	96%	90%	92%
Khaled Sherif	91%	78%	-
Mahmoud Tolba	97%	84%	-
Al3xrods	84%	76%	-
Arpan Chatterjee	-	86%	74%
Güldeniz Bektaş	99%	83%	-

Tabella 1.1. In tabella sono riportati gli autori e le performance dei loro modelli, ordinata per il valore della colonna "Test" in ordine decrescente. I risultati indicano il valore di Accuracy ottenuto durante l'addestramento (Train), la convalida (Validation) e il test (Test), relativi a una singola esecuzione del modello. I risultati del progetto che verrà illustrato in queste pagine, evidenziati in grassetto, sono la media di 7 esecuzioni di test con 20 epoche.

Oltre ai modelli di classificazione, è stato condotto un solo lavoro sulla segmentazione (aggiornato al 20/11/2024). In particolare, l'autore Shrijeeth S. ha presentato un modello di segmentazione basato su SAM (Segment Anything Model), una rete avanzata che utilizza prompt (input specifici come punti o contorni) per segmentare automaticamente qualsiasi area o oggetto richiesto nell'immagine, ottenendo uno score (che nel modello SAM rappresenta il livello di confidenza del modello per

ciascuna maschera) di circa 0,95 su una scala che va da 0 a 1.

Il modello CNN proposto in questo progetto si differenzia da SAM poiché non richiede input specifici per indicare la posizione del tumore. Dopo un addestramento mirato su immagini di tumori, è in grado di individuare autonomamente la regione tumorale, rendendo il processo di segmentazione completamente automatico.

Dunque, come detto in precedenza, i modelli di classificazione sono piuttosto sviluppati, mentre l'interesse per lo sviluppo di modelli di segmentazione automatica, come quello descritto nei capitoli successivi, è attualmente in aumento.

1.3 Il medulloblastoma

1.3.1 Storia del disturbo

Il medulloblastoma è un tumore tipicamente localizzato nel verme cerebellare e ha una tendenza a metastatizzare, colpendo prevalentemente bambini di sesso maschile. Negli anni, i trattamenti sono migliorati significativamente. Inizialmente, la resezione chirurgica era l'unica opzione, con una mortalità post-operatoria elevata. Successivamente, l'introduzione della radioterapia craniospinale ha aumentato la sopravvivenza, anche se con effetti collaterali come problemi neurocognitivi e disfunzioni endocrine. Negli anni '70, la chemioterapia è diventata parte del trattamento standard.

Dal punto di vista dell'imaging e dell'analisi automatica, come in questo progetto di classificazione tra medulloblastoma e immagini non tumorali, queste evoluzioni terapeutiche hanno sottolineato l'importanza di tecnologie che migliorano l'identificazione precoce.

1.3.2 Diagnosi differenziale

Il medulloblastoma presenta caratteristiche di imaging distintive alla risonanza magnetica (RM) e alla tomografia computerizzata (TC), fondamentali per la diagnosi differenziale. Nei bambini, la diagnosi differenziale di una massa nella fossa posteriore include tumori come l'ependimoma, il tumore teratoide/rabdoide atipico e l'astrocitoma pilocitico, mentre negli adulti prevale la malattia metastatica.

Alla TC, il medulloblastoma appare come una massa iperdensa che origina dal verme cerebellare e può causare idrocefalo ostruttivo. Alla RM, le immagini pesate in T1 mostrano tipicamente una lesione ipointensa rispetto alla materia grigia, con un potenziamento eterogeneo del contrasto. Le immagini T2, invece, rivelano spesso una lesione isointensa o iperintensa, con cisti e necrosi. Nel progetto, queste informazioni sono cruciali per l'addestramento del modello di classificazione, che distingue il medulloblastoma dalle immagini senza tumore grazie a queste caratteristiche radiologiche specifiche.

1.3.3 Diffusione e imaging

Il medulloblastoma può diffondersi attraverso il liquido cerebrospinale, un fluido che circonda il cervello e il midollo spinale. Circa un terzo dei pazienti presenta metastasi, cioè la diffusione del tumore, già al momento della diagnosi. Per questo motivo, è fondamentale eseguire un imaging completo del cervello e della colonna vertebrale per ottenere una diagnosi accurata e comprendere l'estensione della malattia.

Nella fase di segmentazione, una delle principali sfide è identificare con precisione le aree tumorali e come si stanno diffondendo. Le tecniche di risonanza magnetica (RM) con pesature T1, T2 e T1C+ sono utili perché aiutano a visualizzare elementi come cisti e necrosi (tessuti morti), oltre a indicare quanto è limitata la diffusione del tumore. Il modello deve quindi riconoscere e tracciare queste caratteristiche.

1.3.4 Caratteristiche radiologiche

Quando si utilizza la tomografia computerizzata (TC), i medulloblastomi appaiono come masse dense con segni di necrosi e idrocefalo ostruttivo (accumulo di liquido). Nella risonanza magnetica, invece, i tumori sono di solito più scuri nelle immagini T1 e di un colore simile o più chiaro nelle immagini T2, mostrando un aumento di segnale dopo l'iniezione di un mezzo di contrasto. La spettroscopia RM può anche rivelare alti livelli di colina e bassi livelli di creatina, informazioni utili per differenziare il medulloblastoma da altri tumori.

Queste caratteristiche sono state utilizzate per costruire il dataset di addestramento del modello, aiutandoci a classificare correttamente le immagini in base a queste peculiarità e a migliorare la precisione delle previsioni.

1.3.5 Aspetti istologici

Il medulloblastoma è considerato un tumore di grado IV e presenta diverse varianti istopatologiche, tra cui quella classica, desmoplastica/nodulare e a grandi cellule/anaplastica. Le varianti desmoplastiche/nodulari tendono ad avere una prognosi migliore rispetto alla variante classica, mentre quelle a grandi cellule e anaplastiche sono associate a una prognosi più sfavorevole.

Queste varianti possono influenzare l'aspetto del tumore nelle immagini e, di conseguenza, la precisione della segmentazione e della classificazione.

Tuttavia, il sito del dataset non forniva informazioni dettagliate sulle varianti istopatologiche presenti nelle immagini, limitando le possibilità di correlare i dati con specifiche caratteristiche cliniche o diagnostiche.

Capitolo 2

Metodo

2.1 Introduzione

In questo capitolo viene fornita una descrizione teorica delle fasi principali del progetto, accompagnata dall'analisi delle scelte metodologiche applicate. Verrà dettagliatamente argomentato, passo dopo passo, il motivo per cui determinate decisioni progettuali sono state preferite ad altre, giustificando le selezioni tecniche e operative in base agli obiettivi del progetto e alle sfide incontrate lungo il percorso.

2.2 Machine learning

L'evoluzione del machine learning ha portato a sviluppi significativi nel campo della classificazione e segmentazione delle immagini mediche. Dagli anni '50, con i primi esperimenti sul perceptron di Frank Rosenblatt, fino agli anni '80 con l'introduzione della retropropagazione (Backpropagation) da parte di Seppo Linnainmaa e Paul Werbos, le reti neurali hanno progressivamente migliorato la capacità di apprendere da dati complessi. La retropropagazione ha permesso l'addestramento efficace di reti neurali a più strati, superando le limitazioni dei modelli lineari.

Negli anni recenti, l'emergere delle reti neurali profonde, o deep learning, ha rivoluzionato l'analisi delle immagini mediche. Tecniche come le reti convoluzionali (CNN) e architetture specializzate, come la U-Net, hanno reso possibile l'automazione della segmentazione tumorale con maggiore accuratezza. Questo è particolarmente rilevante per la segmentazione dei medulloblastomi, dove la capacità di identificare dettagli fini e distinti nelle immagini di risonanza magnetica è cruciale per una diagnosi accurata.

L'uso di tecniche di data augmentation e regolarizzazione migliora ulteriormente le performance dei modelli, riducendo il rischio di overfitting e garantendo una maggiore robustezza nelle predizioni, rendendo il deep learning uno strumento essenziale nella diagnostica dei tumori cerebrali.

Le applicazioni del machine learning si estendono in diversi ambiti, tra cui la computer vision, la predizione, l'analisi semantica, l'elaborazione del linguaggio naturale e il recupero delle informazioni.

Analisi semantica, elaborazione del linguaggio naturale e recupero delle informazioni sono ambiti che mirano a interpretare il significato di testi e dati strutturati,

permettendo a computer di elaborare e recuperare informazioni da documenti e database complessi.

Invece Computer Vision, che è il campo d'interesse in questo studio, comprende il riconoscimento e la rilevazione di oggetti, insieme al loro processamento, ed è uno dei settori chiave in cui il machine learning è stato ampiamente utilizzato. Predizione riguarda invece la classificazione, l'analisi e la raccomandazione, con esempi applicativi che spaziano dalla classificazione di testi e immagini alla diagnosi medica, come nel caso delle analisi di immagini per il riconoscimento di tumori cerebrali.

Queste tecniche sono particolarmente rilevanti per il riconoscimento e la segmentazione dei medulloblastomi, dove l'accuratezza nell'elaborazione delle immagini mediche è cruciale per diagnosi e interventi clinici più efficaci.

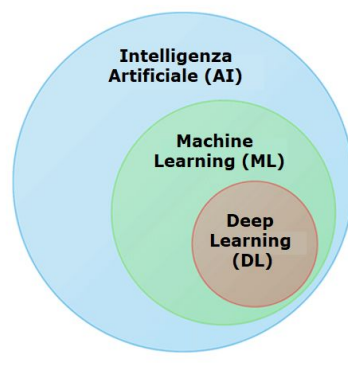


Figura 2.1. Differenze tra AI, Machine Learning e Deep Learning. Fonte: Emc Elettronica (2023).

2.3 Deep learning

Il deep learning, una sottoclasse del machine learning, utilizza reti neurali con molti strati e parametri. È noto anche come deep neural network, poiché la maggior parte dei metodi di deep learning si basa su architetture di reti neurali. In particolare, il deep learning utilizza strati multipli di unità di elaborazione non lineare per l'estrazione e la trasformazione delle caratteristiche. I livelli inferiori, vicini ai dati di input, apprendono caratteristiche semplici, mentre i livelli superiori apprendono caratteristiche più complesse. Questo rende il deep learning particolarmente adatto all'analisi di grandi quantità di dati eterogenei.

Numerosi sviluppi hanno contribuito all'evoluzione delle reti neurali, come le reti neurali auto-organizzanti degli anni '80, il neocognitron di Kunihiko Fukushima per il riconoscimento di caratteri e modelli, e le Convolutional Neural Networks (CNN) introdotte successivamente. Tra le altre architetture rilevanti ci sono le Restricted Boltzmann Machines, sviluppate da Geoffrey Hinton, le Recurrent Neural Networks (RNN), e le Long Short-Term Memory (LSTM), che gestiscono sequenze di dati e sono utilizzate per processi temporali complessi.

Le CNN, come la famosa architettura LeNet di LeCun del 1998, hanno trovato applicazioni nel riconoscimento ottico dei caratteri (OCR), mentre i progressi più recenti includono le Generative Adversarial Networks (GAN) sviluppate da Goodfellow nel 2014, e le Capsule Neural Networks, introdotte da Hinton. L'evoluzione delle reti neurali è stata favorita dall'aumento delle capacità di elaborazione dei chip, dai costi ridotti dell'hardware e dai progressi nella ricerca sull'elaborazione dei segnali e delle informazioni.

Tra le tecniche tipiche di deep learning vi sono gli autoencoder (AE), le Deep Belief Networks (DBN), le CNN, le RNN e il reinforcement learning profondo. Diverse piattaforme e framework, come TensorFlow, PyTorch e Caffe, permettono ai ricercatori di implementare queste tecniche su diverse piattaforme e applicazioni.

Questi progressi, in particolare l'uso di CNN, risultano particolarmente utili nella segmentazione dei tumori cerebrali, dove il deep learning può migliorare l'accuratezza e l'efficacia nel riconoscimento e nell'analisi delle immagini mediche.

2.3.1 Applicare deep learning ad immagini mediche

L'imaging medico è un componente cruciale della sanità moderna, fornendo informazioni essenziali per la diagnosi, il trattamento e il monitoraggio di diverse patologie. Tradizionalmente, le tecniche di analisi delle immagini si sono basate su caratteristiche progettate manualmente e sull'esperienza degli esperti, un processo che può richiedere tempo e essere soggetto a errori umani. Negli ultimi anni, i metodi di machine learning (ML) sono stati sempre più applicati all'analisi delle immagini mediche per migliorare l'efficienza e ridurre gli errori. Tecniche come Support Vector Machines (SVM), alberi decisionali e foreste casuali hanno avuto successo in compiti come la segmentazione delle immagini, il rilevamento di oggetti e la classificazione delle malattie. Tuttavia, tali metodi richiedono la selezione manuale delle caratteristiche, ovvero un processo in cui un esperto umano identifica quali caratteristiche del dataset sono più rilevanti per lo studio.

Con lo sviluppo del deep learning (DL), c'è stato un passaggio significativo verso l'utilizzo di reti neurali profonde in grado di apprendere automaticamente le caratteristiche dalle immagini. A differenza dei metodi ML tradizionali, i modelli DL, come le reti neurali convoluzionali (CNN), sono particolarmente efficaci nell'analisi delle immagini grazie alla capacità di catturare schemi spaziali locali e apprendere rappresentazioni gerarchiche. Le tecniche DL stanno rivoluzionando la ricerca sull'imaging medico, automatizzando l'analisi delle immagini e assistendo nella diagnosi precoce delle malattie, ottimizzando i flussi di lavoro clinici e contribuendo alla medicina personalizzata.

Nell'ambito dell'imaging medico, il DL è stato applicato con successo a diverse modalità, tra cui la risonanza magnetica (MRI), la tomografia computerizzata (CT) e la tomografia a coerenza ottica (OCT), migliorando la segmentazione, la classificazione e la ricostruzione delle immagini. Inoltre, metriche di valutazione come le curve ROC e le matrici di confusione vengono comunemente utilizzate per valutare le prestazioni dei modelli DL nella diagnosi medica, fornendo una misurazione accurata delle loro capacità discriminative e della precisione delle previsioni.

Nonostante i promettenti risultati del deep learning (DL) nell'imaging medico, restano numerose sfide ancora davanti. Una delle principali è la limitata disponibilità di dataset annotati, a causa del processo costoso e dispendioso di annotazione manua-

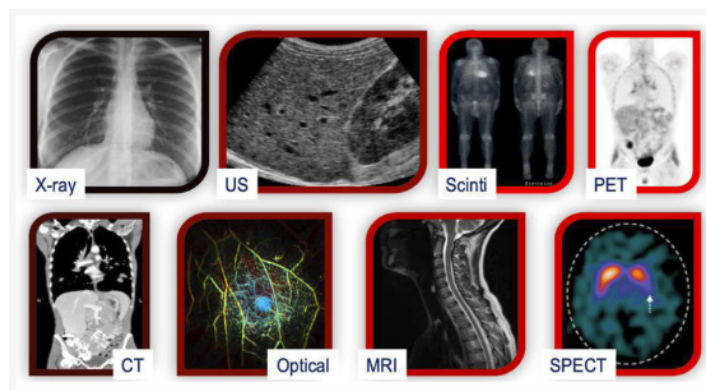


Figura 2.2. Un confronto tra diverse modalità di imaging medico: Scinti (scintigrafia), SPECT (tomografia computerizzata a emissione di fotone singolo), imaging ottico, PET (tomografia a emissione di positroni), TC (tomografia computerizzata), ultrasuoni (US), e MRI (risonanza magnetica).

le. Inoltre, le preoccupazioni sulla privacy dei dati e la condivisione di informazioni sensibili dei pazienti ostacolano la creazione di grandi dataset multi-istituzionali. Un'altra sfida riguarda l'interpretabilità dei modelli DL, che spesso agiscono come "scatole nere", rendendo difficile comprendere il processo decisionale. È cruciale garantire la spiegabilità e l'affidabilità di questi modelli per una loro adozione nella pratica clinica, poiché i medici devono capire le ragioni alla base delle predizioni ed essere in grado di spiegare quali sono le informazioni alla base delle loro decisioni.

Tuttavia, nonostante questi ostacoli, il DL offre numerose opportunità per migliorare l'assistenza sanitaria e i risultati clinici. Con ulteriori ricerche, collaborazioni interdisciplinari e lo sviluppo di algoritmi più sofisticati, il DL ha il potenziale di rivoluzionare l'imaging medico e contribuire significativamente al futuro della medicina.

Le tecniche di deep learning nell'imaging medico svolgono una vasta gamma di funzioni, sia nell'acquisizione delle immagini che nell'identificazione delle patologie. Ad esempio, le reti neurali convoluzionali (CNN) possono essere utilizzate per migliorare la risoluzione delle immagini ottenute da scanner MRI, facilitando una visualizzazione più chiara delle patologie. Inoltre, le CNN sono particolarmente efficaci nell'analisi post-acquisizione, identificando caratteristiche che potrebbero indicare specifiche patologie. Questa doppia funzionalità—miglioramento dell'acquisizione delle immagini e assistenza nell'identificazione delle patologie—è una delle forze chiave delle tecniche di deep learning nell'imaging medico.

Tra le principali tecniche di DL utilizzate nell'imaging medico troviamo le CNN, le reti neurali ricorrenti (RNN) e le reti generative avversarie (GAN). Le CNN, in particolare, sono progettate per compiti di analisi delle immagini. Sono costituite da più livelli—convoluzionali, di pooling e completamente connessi—che collaborano per apprendere rappresentazioni gerarchiche delle immagini di input. I livelli convoluzionali estraggono caratteristiche locali, come bordi e trame, mentre i livelli di pooling riducono le dimensioni spaziali delle mappe di caratteristiche, migliorando l'efficienza e riducendo il rischio di overfitting. Infine, i livelli completamente connessi integrano le caratteristiche locali in pattern globali per eseguire compiti come la classificazione delle immagini.

Le CNN, come LeNet, AlexNet, VGGNet, ResNet e DenseNet, hanno dimostrato grande efficacia in vari compiti di imaging medico, tra cui segmentazione, classificazione e rilevamento di immagini. Un'altra tecnica rilevante è il transfer learning, che consente di utilizzare modelli pre-addestrati su grandi dataset, come ImageNet, e di adattarli a specifici compiti di imaging medico, riducendo il problema della scarsità di dati annotati. È stato impiegato con successo in applicazioni come la diagnosi della retinopatia diabetica e la segmentazione dei tumori cerebrali.

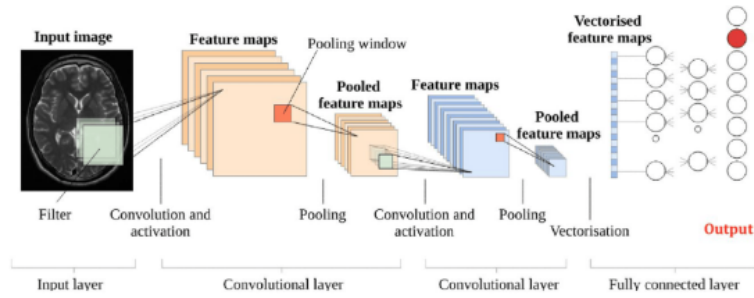


Figura 2.3. Un esempio di architettura CNN per la segmentazione di immagini MRI. La figura illustra il livello di input (immagine medica), i livelli convoluzionali, i livelli di pooling, i livelli completamente connessi e il livello di output, che fornisce i risultati di classificazione o segmentazione.

Le RNN, pur meno diffuse nell'imaging medico, hanno mostrato potenziale nell'analisi di dati sequenziali. Le RNN, combinate con CNN, sono state applicate nella generazione di report radiologici e nell'analisi di video medici, come endoscopie, consentendo il tracciamento degli strumenti chirurgici e l'analisi dei flussi di lavoro.

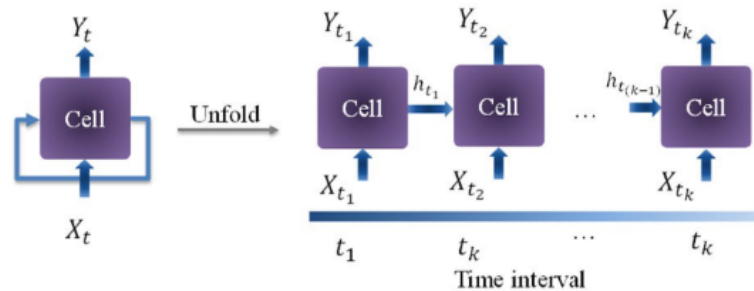


Figura 2.4. Esempio di architettura RNN.

Le RNN sono progettate per lavori in cui è importante mantenere l'ordine dell'input (come ad esempio nell'analisi di una frase, in particolare per la traduzione). Infatti, come si può vedere dalla Figura 2.4, ad ogni passo di un determinato intervallo di tempo t , composto da $t_1, \dots, t_k$, la cellula riceve sia il nuovo input che lo stato nascosto (output) della cellula precedente. In questo modo, la rete "memorizza" informazioni relative ai passi precedenti. Perciò, considerando il vettore di input nell'intervallo t , l'output ritorna come input dello stesso vettore, formando un ciclo, da cui deriva il nome di Reti Neurali Ricorrenti.

Per il progetto sono state utilizzate delle CNN che verranno analizzate nelle prossime sezioni.

2.4 Librerie per l'Elaborazione e il Caricamento dei Dati

2.4.1 Sklearn

Scikit-learn è una libreria open source per l'apprendimento automatico in Python, molto popolare per la sua integrazione con il linguaggio. Tra le sue caratteristiche principali troviamo:

- **Ampia copertura:** Include un'ampia gamma di algoritmi, selezionati tramite revisione comunitaria.
- **Efficienza:** Gli algoritmi sono implementati con librerie binarie ottimizzate, migliorando le prestazioni.
- **Supporto della comunità:** La documentazione è ben mantenuta e ci sono processi rigorosi per il monitoraggio dei bug.
- **Convenzione unificata:** Adozione di standard coerenti per input/output, facilitando il passaggio tra metodi.

Inoltre, Scikit-learn si occupa di trasformazione dei dati, apprendimento supervisionato, apprendimento non supervisionato e valutazione dei modelli. I dettagli di ogni argomento possono essere trovati nella guida utente sul sito web Scikit-learn (http://scikit-learn.org/stable/user_guide.html)

2.4.2 Kares e TensorFlow

Scikit-learn può essere usato per addestrare alcune tipologie di reti neurali artificiali (ANN), ma presenta limitazioni significative per il deep learning, quali:

1. L'addestramento di reti neurali profonde richiede la regolazione di molti parametri e iperparametri, un processo computazionalmente intensivo che spesso necessita di GPU o TPU per calcoli paralleli, non supportati attualmente da scikit-learn.
2. Scikit-learn non include implementazioni di alcune architetture di reti popolari, come le reti neurali convoluzionali (CNN) e le reti neurali ricorrenti (RNN).

Perciò, si preferisce utilizzare Keras con backend TensorFlow, progettato per addestrare varie tipologie di ANN e compatibile con hardware come CPU, GPU e TPU. Keras è una API di deep learning scritta in Python, nota per la semplicità e l'efficienza, ed è considerata "il Python del deep learning" grazie alla sua popolarità tra i data scientist. Originariamente sviluppata per supportare Theano, Keras si è poi evoluta per includere TensorFlow e altri backend come CNTK e MXNet. Dal 2018, TensorFlow ha adottato Keras come sua API di alto livello, rendendo Keras la scelta principale per costruire e addestrare modelli di deep learning complessi.

2.4.3 Matplotlib

Matplotlib è una delle librerie di visualizzazione dei dati più popolari in Python. Creata da John Hunter e sostenuta da numerosi collaboratori, è diventata uno strumento fondamentale per scienziati e filosofi in tutto il mondo. Questa libreria offre un pacchetto grafico integrato per la visualizzazione dei dati e si integra facilmente con altre librerie come NumPy e Pandas, rappresentando un elemento chiave nello stack della scienza dei dati in Python.

2.4.4 Seaborn

Seaborn è una libreria di visualizzazione grafica che si basa sulle funzionalità di Matplotlib. Essa semplifica l'accesso a diverse tecniche di visualizzazione dei dati, come la mappatura dei colori a variabili specifiche e l'uso del faceting. Seaborn è progettata per lavorare in modo più fluido con i DataFrame di Pandas, rendendo più semplici le operazioni di visualizzazione dei dati.

2.4.5 Pandas

Pandas è una libreria open-source in Python che offre strutture dati e strumenti di analisi dei dati avanzati. Questa libreria è utilizzata in vari settori, inclusi scienza dei dati, modellazione computazionale, finanza, economia, analisi statistica e apprendimento automatico. Pandas è noto per la sua facilità d'uso e le sue prestazioni elevate, rendendolo uno strumento essenziale per i programmatori Python.

2.4.6 NumPy

NumPy, abbreviazione di Numerical Python, è una libreria per l'analisi dei dati e il calcolo numerico che include oggetti array multidimensionali e una serie di routine per il loro trattamento. Grazie a NumPy, data scientist e programmatori possono eseguire operazioni matematiche, numeriche e logiche sugli array, contribuendo a costruire modelli computazionali efficienti per la scienza e la matematica.

Queste librerie rappresentano strumenti fondamentali per l'analisi e la visualizzazione dei dati in Python, ognuna con le proprie specificità e vantaggi.

2.5 Reti Neurali

2.5.1 Neurone Biologico

Le reti neurali biologiche sono costituite da miliardi di neuroni, ciascuno formato da un corpo cellulare, dendriti e assone. I dendriti ricevono segnali da altri neuroni tramite le sinapsi, che trasmettono questi impulsi al corpo cellulare e, successivamente, all'assone verso altri neuroni. Nel cervello umano, ogni neurone è in media collegato a migliaia di altri, creando un sistema altamente interconnesso capace di elaborare informazioni complesse, apprendere, memorizzare, coordinare movimenti e supportare funzioni cognitive ed emotive.

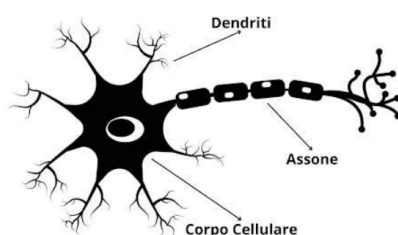


Figura 2.5. Struttura del neurone biologico

I neuroni hanno due stati principali: lo stato di riposo e lo stato attivo. In stato di riposo, il potenziale di membrana è stabile e il neurone non trasmette segnali. Quando il potenziale di membrana si modifica diventando meno negativo, il neurone può raggiungere una soglia critica e generare un impulso elettrico lungo l'assone, noto come potenziale d'azione. Questo impulso provoca il rilascio di neurotrasmettitori nelle sinapsi, che possono inibire o stimolare i dendriti del neurone successivo. Questo processo di adattamento, chiamato plasticità sinaptica, è cruciale per l'apprendimento, la memoria e la capacità del cervello di recuperare da lesioni e adattarsi a nuovi ambienti.

2.5.2 Neurone Artificiale

Uno dei modelli più noti per descrivere matematicamente un neurone biologico è il *perceptron*. Questo modello rappresenta l'unità base di elaborazione in una rete neurale artificiale, rinomato per la sua semplicità e versatilità nel modellare relazioni tra input e output. Sebbene il perceptron singolo sia limitato nella capacità di risolvere problemi complessi, se combinato in una struttura multistrato, può affrontare anche sfide di intelligenza artificiale complesse.

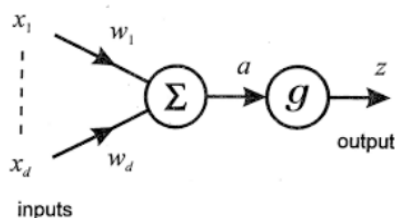


Figura 2.6. Modello di *Perceptron*

Il modello si compone di due fasi principali. Nella prima fase, si calcola la somma pesata di tutti gli ingressi $(x_1, x_2, \dots, x_d)$ usando i pesi $(w_1, w_2, \dots, w_d)$. Questa somma è definita come:

$$a = \sum_{i=1}^d w_i x_i + w_0$$

dove x_i rappresenta l'input, w_i il peso associato a ciascun input, e w_0 è il *bias*, o soglia di attivazione del neurone. Assegnando un valore costante $x_0 = 1$, si ottiene una versione più compatta:

$$a = \sum_{i=0}^d w_i x_i$$

I pesi possono essere positivi o negativi a seconda che si desideri un effetto eccitatorio o inibitorio. Nella seconda fase, si calcola l'uscita finale z del neurone applicando una funzione di attivazione, che trasforma la somma a in un valore di output:

$$z = g\left(\sum_{i=0}^d w_i x_i\right)$$

La funzione di attivazione è importante perché restituisce un valore di output limitato a un certo intervallo, dato che la somma ponderata può produrre qualsiasi valore. Inoltre, è fondamentale che la funzione di attivazione sia non lineare, in modo da permettere alla rete neurale di apprendere e generalizzare le informazioni. A seconda del tipo di strato della rete, è possibile scegliere diverse funzioni di attivazione. Tra le più comuni troviamo:

- **Sigmoide**

$$g(a) = \frac{1}{1 + e^{-a}}$$

- **Tangente iperbolica**

$$g(a) = \tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}$$

- **ReLU (Rectified Linear Unit)**

$$g(a) = \max(0, a)$$

- **Soglia**

$$g(a) = \begin{cases} 1, & \text{se } a > s \\ 0, & \text{se } a \leq s \end{cases}$$

dove s rappresenta il valore soglia.

La funzione sigmoide è molto utilizzata nelle reti neurali per la sua non linearità e output limitato, rendendola adatta per problemi di classificazione binaria, poiché restituisce un valore compreso tra 0 e 1 che può essere inteso come la probabilità per una delle due classi. Invece, nelle reti neurali profonde, la funzione ReLU è preferita per l'addestramento di modelli complessi, migliorando le prestazioni. Infine, il perceptron classico utilizza la funzione di attivazione a soglia.

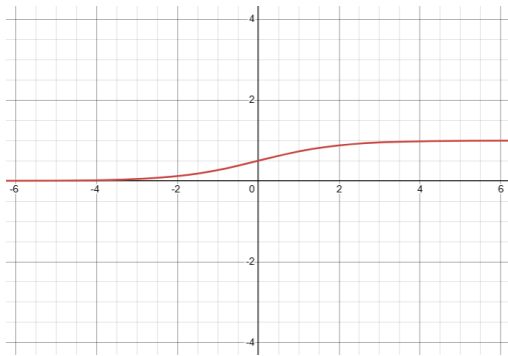


Figura 2.7. Funzione Sigmoidea (generata con Desmos)

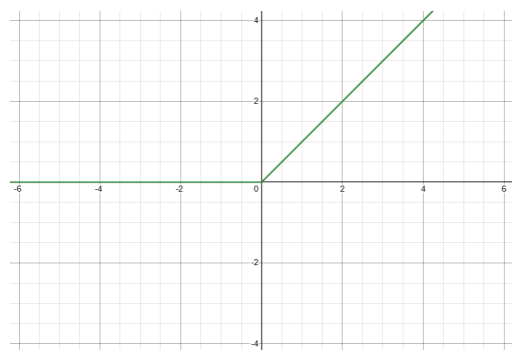


Figura 2.8. Funzione ReLU (generata con Desmos)

2.6 Reti neurali Artificiali

Dunque i concetti fondamentali di una rete neurale artificiale sono:

1. **Neuroni:** I neuroni rappresentano le unità fondamentali di elaborazione all'interno di una rete neurale. In base alla loro posizione nei vari strati, ricevono input, eseguono calcoli matematici specifici e trasmettono il risultato ai neuroni successivi o all'output finale.
2. **Pesi (w):** Ogni input che entra in un neurone ha un peso associato, che ne determina l'importanza relativa rispetto agli altri. Durante il processo di addestramento, questi pesi vengono regolati per migliorare la correlazione con la variabile target, assegnando valori maggiori ai parametri più rilevanti.
3. **Bias (b):** Simile all'intercetta c nelle equazioni lineari ($y = mx + c$), il bias è una costante che permette al modello di adattarsi meglio ai dati, evitando che il modello si concentri solo sull'origine (ossia quando $c = 0$).
4. **Funzione di attivazione $g(x)$:** Presente in ogni neurone, decide se il neurone stesso si attiverà per trasmettere il segnale successivo. Le funzioni di attivazione permettono di introdurre la non linearità necessaria alla rete per imparare relazioni complesse.

Le reti neurali artificiali (ANN, dall'inglese *Artificial Neural Networks*) sono modelli computazionali ispirati alle reti neurali biologiche. Le reti neurali feedforward multilayer (MLP, ovvero Multilayer Perceptron) sono una classe specifica di reti neurali che utilizzano il perceptron come unità di base, organizzata su più strati, sono una tipologia di ANN, elaborano un insieme di input indipendenti $x = (x_1, x_2, \dots, x_d)$ e producono un output di variabili dipendenti $y = (y_1, y_2, \dots, y_c)$. Gli output della rete sono definiti da funzioni il cui comportamento è determinato da parametri, noti come pesi $w = (w_1, w_2, \dots, w_d)$.

Queste reti sono suddivise in strati per facilitare l'apprendimento di strutture complesse. Ogni strato è composto da neuroni interconnessi e svolge un ruolo specifico:

- **Strato di ingresso:** riceve i dati in input, dove ogni neurone rappresenta una caratteristica, come un singolo pixel di un'immagine.

- **Strato nascosto:** estrae caratteristiche rilevanti e cattura relazioni complesse tra i dati, rappresentando l'essenza del processo di apprendimento.
- **Strato d'uscita:** fornisce l'output finale della rete, che può variare in base al compito, come una probabilità, una classificazione, o un valore numerico.

Questa struttura gerarchica consente alle reti neurali di affrontare e risolvere numerosi problemi complessi, adattandosi efficacemente a diversi contesti di apprendimento.

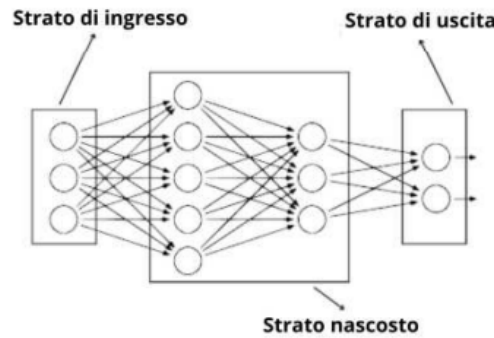


Figura 2.9. Modello Multilayer Perceptron (MLP)

2.6.1 Processo di Addestramento

Per utilizzare un modello neurale, è necessario avviare il processo di addestramento. Durante questa fase, la rete acquisisce conoscenze dai dati di input per generalizzare e produrre risultati accurati su nuovi dati. In particolare, vengono determinati i pesi $\mathbf{w}$ delle connessioni tra i neuroni con l'obiettivo di ridurre l'errore tra l'output della rete e il risultato desiderato.

Per il processo di addestramento, si utilizza un insieme di dati chiamato *training set*, composto da coppie $(\mathbf{x}_q, \mathbf{t}_q)$ per $q = 1, \dots, n$, dove $\mathbf{x}_q$ rappresenta l'input e $\mathbf{t}_q$ l'output target. Il processo di addestramento è diviso in tre fasi principali:

- **Fase 1 - Propagazione dell'input:** Si inizializzano i pesi, generalmente a valori casuali, e si propagano gli input $\mathbf{x}$ attraverso gli strati della rete. Gli input vengono pesati da connessioni sinaptiche $\mathbf{w}$ e sommati insieme. Per un neurone generico j , il valore di output y_j è dato da:

$$y_j = g \left(\sum_{i=0}^n w_{ij} x_i \right) \quad (2.1)$$

dove g è la funzione di attivazione. Qui, x_0 rappresenta il bias, solitamente con valore pari a 1 e w_{ij} rappresenta il peso della connessione tra l'input x_i e il neurone j nello strato corrente.

- **Fase 2 - Calcolo del costo:** La funzione di perdita (loss function) o di costo, comunemente la cross-entropia per classificazione, viene utilizzata per regolare i pesi del modello durante l'addestramento. L'obiettivo è minimizzare questa perdita: minore è la perdita, migliore è il modello. Un modello perfetto

avrebbe una perdita di cross-entropia pari a zero. La cross-entropia misura la differenza tra le distribuzioni di probabilità scoperte e quelle previste dal modello. Per due distribuzioni di probabilità P (target) e Q (approssimata dal modello), la cross-entropia è definita come:

$$H(P, Q) = - \sum_{x \in X} P(x) \cdot \log(Q(x)) \quad (2.2)$$

dove X rappresenta l'insieme di tutte le classi possibili.

- **Fase 3 - Retropropagazione dell'errore:** L'errore viene retropropagato attraverso gli strati interni della rete, aggiornando i pesi tramite l'algoritmo di *backpropagation* e la discesa del gradiente. La retropropagazione calcola i gradienti della funzione di costo rispetto ai pesi della rete tramite la catena derivata, e la discesa del gradiente utilizza questi gradienti per aggiornare i pesi riducendo progressivamente la funzione di costo.

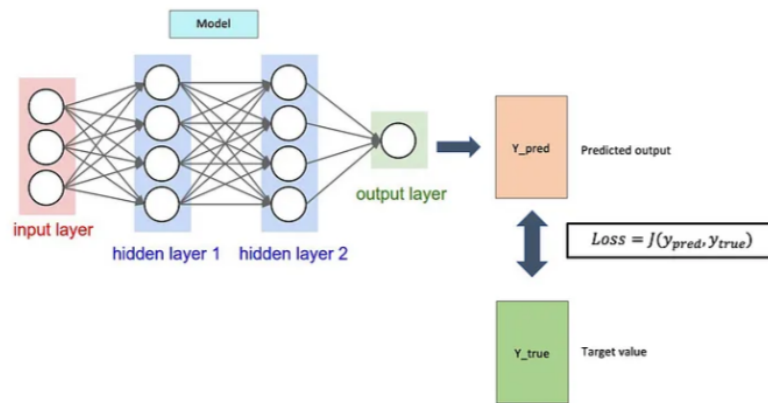


Figura 2.10. Funzioni di perdita basate sulla cross-entropia. Fonte: *Understanding Loss Functions for Classification* di Nghi Huynh.

2.7 Backpropagation

La retropropagazione è il meccanismo di apprendimento che consente al Perceptron Multilayer di adeguare iterativamente i pesi all'interno della rete, con l'obiettivo di minimizzare la funzione di costo.

Per garantire il corretto funzionamento della retropropagazione, è fondamentale che la funzione che combina gli input e i pesi in un neurone—ad esempio, la somma pesata—e la funzione di attivazione—come la ReLU—siano differenziabili. La differenziabilità consente infatti di calcolare il gradiente durante il processo di ottimizzazione tramite l'algoritmo di discesa del gradiente, utilizzato nel Perceptron Multilayer.

In ogni iterazione, dopo che le somme pesate sono state propagate attraverso tutti gli strati della rete, viene calcolato il gradiente della funzione di costo (ad esempio, la cross-entropy loss) per tutte le coppie di input e output. Successivamente, il gradiente dell'errore viene retropropagato all'indietro attraverso gli strati della rete, aggiornando i pesi di ogni strato in base al valore del gradiente calcolato per

minimizzare l'errore. Questo processo continua fino a soddisfare un criterio di arresto, come il raggiungimento di un numero massimo di epoche o una variazione minima nella funzione di costo tra iterazioni successive. Se il numero di epoche è troppo basso, si rischia di non minimizzare abbastanza l'errore. Al contrario, se il numero di epoche è troppo alto ad un certo punto smette di migliorare e si potrebbe adattare troppo ai dati utilizzati per la costruzione del modello (overfitting), non riuscendo più a generalizzare su dati nuovi.

2.8 Gradient Descent

Il *gradient descent*, o discesa del gradiente, è un metodo di ottimizzazione numerica utilizzato per trovare il minimo (o massimo) di una funzione. Nel contesto dell'apprendimento automatico e delle reti neurali, questo algoritmo si avvale del vettore del gradiente per identificare il minimo della funzione di costo e aggiornare i valori dei pesi.

Il vettore gradiente è composto dalle derivate parziali della funzione di costo rispetto ai pesi. Ogni posizione del vettore del gradiente corrisponde alla derivata parziale della funzione di costo rispetto a un determinato peso (o bias) del modello. Le derivate indicano l'andamento locale della funzione: se la derivata è positiva, significa che la funzione cresce rispetto a quel parametro; se è negativa, la funzione decresce. Pertanto, ogni componente del vettore gradiente indica quanto e in che direzione quel peso specifico deve essere modificato per ridurre l'errore. Utilizzando l'intero vettore gradiente in un processo iterativo, come la discesa del gradiente, possiamo aggiornare i pesi per minimizzare progressivamente la funzione di costo.

Il processo di discesa del gradiente si articola in diverse fasi, che possono essere riassunte come segue:

1. **Inizializzazione dei pesi:** i pesi vengono inizializzati con valori casuali o scelti in modo appropriato. Questi valori verranno modificati iterativamente durante il processo di addestramento.
2. **Calcolo del gradiente:** si calcola il vettore gradiente, che indica la direzione e l'intensità della variazione della funzione di costo rispetto ai pesi.
3. **Aggiornamento dei pesi:** i pesi della rete vengono aggiornati in base alla direzione fornita dal gradiente. Il gradiente indica come la funzione di costo cambia rispetto ai pesi; quindi, per minimizzare la funzione di costo, i pesi vengono aggiornati sottraendo una frazione del gradiente, scalata dal parametro di apprendimento:

$$w = w - \eta \cdot \nabla L$$

Dove:

- w è il peso attuale.
- η è il parametro di apprendimento (learning rate).
- ∇L è il gradiente della funzione di costo rispetto ai pesi.

Il parametro di apprendimento (o *learning rate*) determina la dimensione dei passi che l'algoritmo compie per minimizzare la funzione di costo. Se questo valore è troppo basso, la convergenza verso la soluzione ottimale sarà molto lenta; se è troppo alto, la soluzione finale potrebbe divergere o oscillare, senza mai arrivare a un minimo.

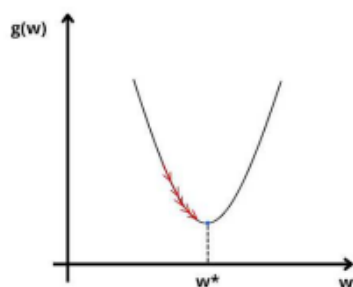


Figura 2.11. Grafico dell'aggiornamento di un singolo peso con un parametro di apprendimento troppo piccolo.

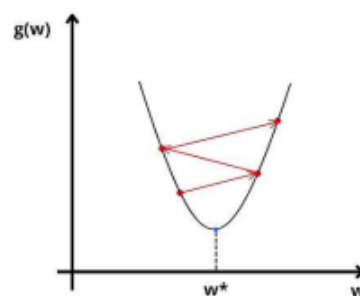


Figura 2.12. Grafico dell'aggiornamento di un singolo peso con un parametro di apprendimento troppo grande.

4. **Ripetizione fino alla convergenza:** i passaggi 2 e 3 vengono ripetuti iterativamente fino a quando la funzione di costo non raggiunge un valore minimo accettabile. È possibile stabilire anche un limite massimo per il numero di iterazioni da eseguire.

Sebbene esistano altre strategie di addestramento, la scelta della procedura da utilizzare dipende dall'applicazione specifica e dalla complessità del problema da risolvere. Tuttavia, la descrizione di queste tecniche non verrà affrontata in questo contesto.

2.9 Modelli Neurali

L'utilizzo di un singolo modello non può essere universalmente adatto per tutte le applicazioni e strutture dati. In questo capitolo analizzeremo le reti convoluzionali e ne discuteremo le caratteristiche e applicazioni specifiche.

2.9.1 Reti Convoluzionali

Le reti convoluzionali, o CNN (Convolutional Neural Networks), rappresentano una classe di reti neurali specializzate nella manipolazione di dati visivi. Questo tipo di rete trae ispirazione dal funzionamento della corteccia visiva umana, che è in grado di estrarre forme e caratteristiche dai dati di ingresso. Le CNN hanno una struttura simile a quella delle reti feedforward, ma una caratteristica distintiva è la presenza dei livelli di convoluzione, che hanno il ruolo di estrarre le *features* dai dati in input.

Le CNN includono strati convoluzionali che applicano filtri per identificare differenti caratteristiche nelle immagini. Nei primi strati, si usano filtri semplici, mentre nei livelli successivi si adottano filtri più complessi. Un filtro è una matrice di valori, appresa durante l'addestramento, che viene fatta scorrere lungo l'immagine di input. In ogni posizione, si genera un valore di output calcolando il prodotto scalare tra il filtro e la porzione di immagine corrispondente. Questa operazione è chiamata *convoluzione*.

Per comprendere meglio, consideriamo una matrice di input I ed un kernel K :

$$I = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}, \quad K = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applicando la convoluzione $I * K$, otteniamo il risultato:

$$z = (0 \cdot -1) + (0 \cdot -1) + \dots + (1 \cdot -1) + (2 \cdot -1) = 4$$

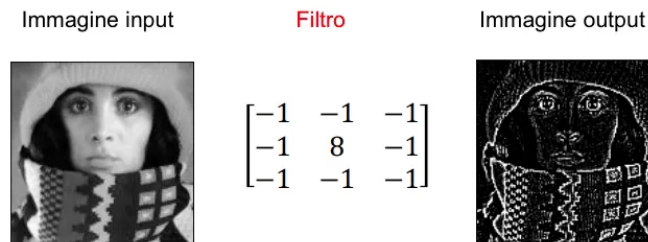


Figura 2.13. Immagine tratta da <https://www.domsoria.com/2019/10/come-funziona-una-rete-neurale-cnn-convolutional-neural-network/>.

Oltre ai livelli di convoluzione, le CNN includono anche strati di *pooling*, che, usati alternativamente alle convoluzioni, permettono la riduzione delle dimensioni dell'immagine, mantenendo le informazioni più rilevanti. Durante il pooling, una finestra rettangolare si sposta sull'immagine e seleziona il valore massimo (max-pooling), minimo (min-pooling) o medio (average-pooling) dei pixel coperti. Questa operazione riduce la risoluzione dell'immagine preservando le caratteristiche estratte nei livelli precedenti.

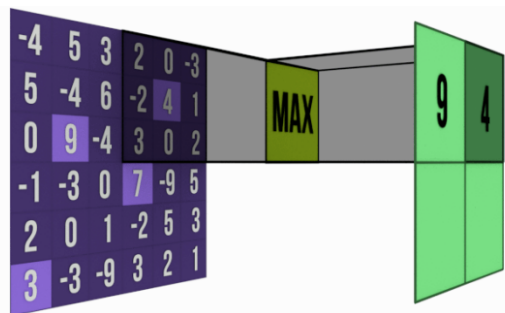


Figura 2.14. Esempio max-pooling. Fonte: <https://www.kaggle.com/code/aayusmaanjan/basics-of-cnn-with-pytorch-lightning>

2.9.1.1 Architettura di una CNN

L'architettura di una tipica CNN è composta da tre principali componenti: lo strato di input, gli strati nascosti e lo strato di output. Lo strato di input riceve l'immagine e la passa agli strati nascosti, formati da molteplici strati convoluzionali e di pooling. Lo strato di output, invece, fornisce l'etichetta della classe predetta o un punteggio di probabilità per ciascuna classe.

Gli strati nascosti sono la parte cruciale di una CNN. Il numero di strati e il numero di filtri in ogni strato possono essere modificati per ottimizzare le prestazioni

della rete. Un'architettura tipica di una CNN è composta da più strati convoluzionali, seguiti da uno o più strati di pooling, e da uno strato completamente connesso, che collega ogni neurone a tutti i neuroni degli strati precedenti e fornisce l'output finale.

2.9.1.2 Funzionamento delle CNN

Una CNN inizia prendendo un'immagine di input, che viene poi trasformata in una *feature maps* (mappa di caratteristiche) attraverso una serie di strati convoluzionali e di pooling. Il livello convoluzionale applica una serie di filtri all'immagine di input, con ciascun filtro che genera una mappa di caratteristiche evidenziando un aspetto specifico dell'immagine. Lo strato di pooling riduce quindi le dimensioni della mappa di caratteristiche, mantenendo le informazioni più importanti.

La mappa delle caratteristiche prodotta dallo strato convoluzionale passa quindi attraverso strati aggiuntivi di convoluzione e pooling, ognuno dei quali apprende caratteristiche sempre più complesse dell'immagine di input. L'output finale della rete è un'etichetta di classe predetta o un punteggio di probabilità per ogni classe, a seconda del compito.

Per le CNN, la funzione di attivazione più comunemente usata è la ReLU (Rectified Linear Unit), scelta per la sua semplicità ed efficienza durante l'addestramento, e che consente attivazioni sparse dei neuroni, riproducendo parzialmente un comportamento simile a quello del cervello umano. Nelle fasi finali, la CNN include solitamente una rete completamente connessa (*fully connected layer*), che analizza le caratteristiche principali estratte e restituisce un risultato finale che può rappresentare una previsione, una classificazione o un'ulteriore elaborazione dei dati, a seconda dell'obiettivo della rete.

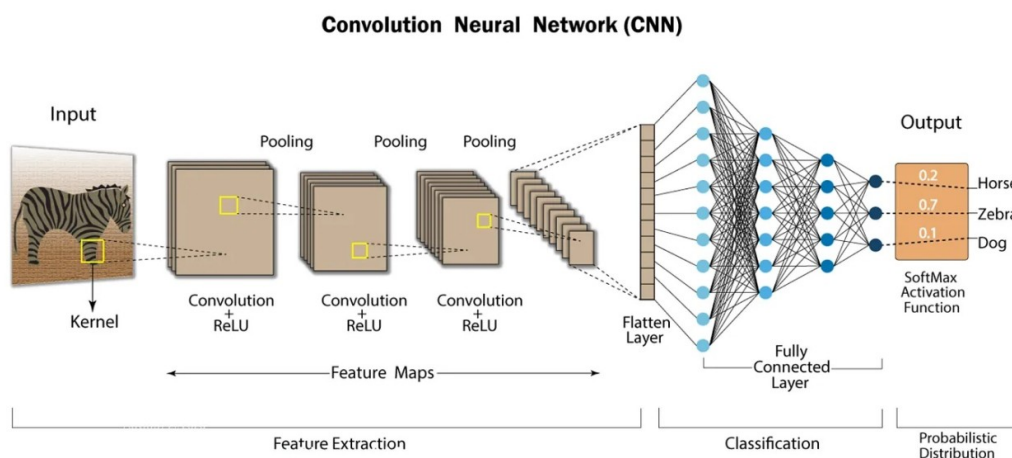


Figura 2.15. Architettura CNN. Immagine tratta da [shahriar2020cnn](#).

2.9.1.3 Applicazioni delle CNN

Le CNN trovano applicazione in una vasta gamma di compiti di visione artificiale, come la classificazione di immagini, il riconoscimento di oggetti, la segmentazione semantica e il trasferimento di stile.

- **Classificazione di immagini:** consiste nell'assegnare un'etichetta di classe a un'immagine di input. Le CNN possono essere addestrate su grandi set di

dati di immagini etichettate per imparare le relazioni tra i pixel dell'immagine e le etichette di classe, e poi applicate a nuove immagini per effettuare una previsione.

- **Riconoscimento di oggetti:** consiste nell'identificare oggetti di una classe specifica in un'immagine di input e nel segnalarne la posizione. Questo può essere utile per applicazioni come la sicurezza e la sorveglianza, dove è importante rilevare e tracciare oggetti in tempo reale.
- **Segmentazione semantica:** consiste nell'assegnare un'etichetta di classe a ciascun pixel di un'immagine di input, producendo un'immagine segmentata utilizzabile per ulteriori analisi. È utile, ad esempio, nell'analisi di immagini mediche per segmentare strutture specifiche da analizzare ulteriormente.

2.9.1.4 Vantaggi delle CNN

- **Estrazione automatica delle caratteristiche:** Le CNN possono estrarre automaticamente caratteristiche rilevanti da un'immagine di input, riducendo la necessità di ingegneria delle caratteristiche manuali.
- **Invarianza spaziale:** Sono in grado di riconoscere oggetti in un'immagine indipendentemente dalla posizione, dimensione o orientamento.
- **Robustezza al rumore:** Le CNN possono gestire immagini rumorose o disordinate, essendo utili per applicazioni reali in cui la qualità dell'immagine può variare.
- **Apprendimento trasferibile:** Le CNN possono sfruttare modelli pre-addestrati, riducendo la quantità di dati e risorse computazionali necessarie per addestrare un nuovo modello.
- **Alte prestazioni:** Le CNN hanno dimostrato prestazioni all'avanguardia in numerosi compiti di visione artificiale, tra cui classificazione, rilevamento di oggetti e segmentazione semantica.

2.9.1.5 Limitazioni delle CNN

- **Costo computazionale:** L'addestramento di una CNN profonda può essere costoso in termini computazionali, richiedendo una quantità significativa di dati e risorse.
- **Overfitting:** Le CNN profonde tendono a soffrire di overfitting, soprattutto quando vengono addestrate su set di dati piccoli.
- **Scarsa interpretabilità:** Le CNN sono spesso considerate "black box", rendendo difficile comprendere il motivo di una previsione specifica.
- **Limitazione a strutture regolari:** Le CNN sono limitate a strutture simili a griglie e non possono gestire forme irregolari o dati non strutturati.

Capitolo 3

Esperimenti

Il progetto si articola in due fasi principali: la classificazione e, successivamente, la segmentazione. In questo capitolo verranno descritte le implementazioni e le scelte effettuate durante lo sviluppo.

3.1 Dataset

Per iniziare l'implementazione del codice è stato necessario selezionare un dataset adeguato. Il requisito principale è stato disporre di un dataset sufficientemente ampio per addestrare il modello in modo efficace. Per i vari motivi elencati nel Capitolo 1, è stato possibile costruire un dataset di 131 immagini di medulloblastomi, recuperate da due pubblicazioni di dataset privati su Kaggle:

- <https://www.kaggle.com/datasets/waseemnagahhenes/brain-tumor-for-14-classes>
- <https://www.kaggle.com/datasets/fernando2rad/brain-tumor-mri-images-44c>

A queste si aggiungono altre 265 immagini MRI del cervello, prive di tumore, anch'esse ricavate dai dataset sopra elencati.

Per l'addestramento sono state selezionate 101 immagini di medulloblastoma (training set), suddivise in sottocartelle a seconda del tipo di MRI:

- **T1**: 13 immagini
- **T1C+**: 57 immagini
- **T2**: 31 immagini

Le restanti 30 immagini sono state utilizzate per la fase di testing del modello (validation set), distribuite in sottocartelle contenenti 10 immagini per ciascuna modalità di MRI.

Per quanto riguarda la segmentazione, sono state selezionate 111 delle 131 immagini di medulloblastoma. Le maschere sono state create manualmente per ciascuna di queste 111 immagini, scartando le restanti 20 poiché, in collaborazione con un medico non specializzato in radiologia, si è deciso di non includerle per evitare valutazioni incerte.

3.1.1 Strumento per la creazione di maschere

Per creare le maschere è stato utilizzato il software **Fiji**, una versione avanzata di ImageJ, comunemente impiegata in bioinformatica e ricerca medica per l'elaborazione e l'analisi delle immagini. Fiji offre un'interfaccia utente intuitiva, che permette di creare e salvare le maschere generate con pochi passaggi, velocizzando così il lavoro. Inoltre, è una piattaforma open source in continuo aggiornamento da parte della comunità scientifica.

3.2 Classificazione

La classificazione di immagini per due classi prevede la previsione dell'appartenenza dell'immagine a una delle due classi, ad esempio "medulloblastoma" e "no_tumor". L'etichetta predetta viene assegnata all'intera immagine, quindi ogni pixel è considerato parte di una singola classe. Questa tecnica è utile quando l'obiettivo è determinare il tipo di contenuto presente nell'immagine nel suo complesso, senza localizzare esattamente le diverse aree o oggetti.

In questa sezione vengono elencati i passaggi principali del processo di classificazione.

3.2.1 Data Augmentation del dataset

Per ovviare al problema del dataset ridotto, si utilizza una tecnica chiamata *data augmentation*. Nel progetto, si sfrutta la classe `ImageDataGenerator` di Keras per implementare una serie di tecniche di *data augmentation* avanzate, applicate al *training set* per migliorare la capacità del modello di generalizzare e ridurre il rischio di *overfitting*, che si verifica quando il modello apprende troppo dettagliatamente i dati di addestramento, inclusi eventuali rumori o errori. Queste trasformazioni includono rotazioni, traslazioni orizzontali e verticali, modifiche di zoom, tagli obliqui (*shear*), variazioni di luminosità e capovolgimenti orizzontali casuali. Le immagini trasformate vengono poi riempite utilizzando l'algoritmo `nearest` per evitare distorsioni ai bordi. Tali tecniche permettono di aumentare virtualmente la dimensione del dataset, esponendo il modello a varianti diverse delle immagini originali.

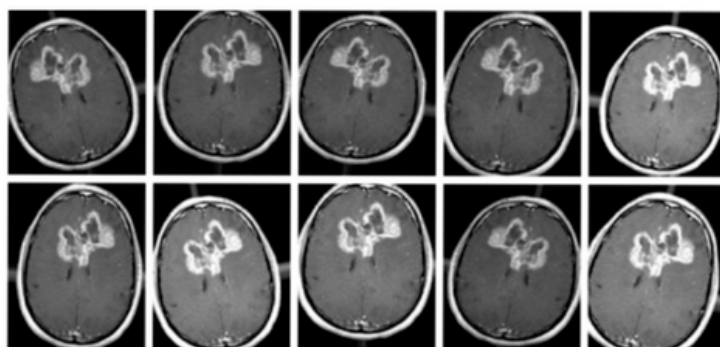


Figura 3.1. Una raccolta di immagini aumentate derivate da una singola immagine.

3.2.2 Pre-processamento

Il *pre-processing*, o pre-elaborazione, è una fase cruciale nella preparazione dei dati destinati all'addestramento di modelli neurali. Utilizzare dati grezzi non elaborati può compromettere la precisione e le prestazioni del modello. Durante la pre-elaborazione, i dati di input vengono trasformati attraverso una serie di passaggi che ne migliorano la qualità e la rilevanza, ottimizzando l'efficacia dell'addestramento. Di seguito vengono illustrate alcune delle tecniche di pre-processing utilizzate.

3.2.2.1 Normalizzazione

La normalizzazione dei dati rappresenta un passaggio essenziale nella fase di pre-elaborazione per l'apprendimento automatico, in quanto regola i valori delle diverse feature. Normalizzare i dati facilita la convergenza durante l'addestramento, specialmente per algoritmi di ottimizzazione basati sul gradiente, come la discesa del gradiente. Se le feature hanno scale diverse, questi algoritmi possono incontrare difficoltà nel trovare la soluzione ottimale. La *normalizzazione* è una tecnica che rende comparabili le caratteristiche dei dati, uniformandole su una scala comune. Per fare ciò, si divide ogni valore dei pixel per 255.

3.2.2.2 Sample-wise Centering e Sample-wise Standardization

Il *sample-wise centering* sottrae ad ogni pixel la media di ciascuna immagine, mentre il *sample-wise standardization* divide ogni pixel per la deviazione standard (indica quanto i dati di un insieme si discostano dalla loro media) della stessa immagine, ottenendo dati con media zero e deviazione standard uno.

3.2.3 Caricamento dei dati

Dopo la fase di pre-processamento è possibile procedere al caricamento dei dati. In particolare il modello ha un *batch size* pari a 16 immagini, ovvero processa 16 campioni (in questo caso immagini) alla volta in fase di addestramento per migliorare l'efficienza e sfruttare al meglio le risorse hardware; un numero troppo alto potrebbe sovraccaricare la memoria. Le 16 immagini in input vengono quindi definite come un batch di immagini. Successivamente, le immagini sono ridimensionate a 224×224 pixel, una dimensione standard necessaria per garantire la compatibilità con il modello pre-addestrato utilizzato nelle fasi successive.

In fase di training, le immagini vengono mescolate a ogni epoca per evitare che il modello impari una specifica sequenza dei dati. Inoltre, viene assegnato un peso pari a 1 alla classe non tumore e 1.3 alla classe medulloblastoma per bilanciare il numero inferiore di immagini di quest'ultima nel dataset.

3.2.4 Modello pre-addestrato

Le immagini, dopo essere state caricate e ridimensionate, vengono fornite in input a un modello pre-addestrato. In ambito di *deep learning*, l'utilizzo di modelli pre-addestrati accelera lo sviluppo di nuove soluzioni, riducendo costi e tempi rispetto alla creazione da zero, e consente di affrontare compiti correlati in modo più efficace. I principali modelli considerati in questo studio sono stati VGG16, InceptionV3 e ResNet50. VGG16 è stato infine scelto come definitivo, poiché ha dimostrato prestazioni migliori.

VGG16

VGG16 è una rete neurale convoluzionale profonda con 16 livelli, addestrata su milioni di immagini del database ImageNet. Questo modello pre-addestrato può distinguere tra 1000 categorie, inclusi animali e oggetti comuni, come mouse e tastiere. Grazie a ciò, VGG16 ha una capacità classificatoria versatile. La rete accetta immagini di risoluzione 224×224 .

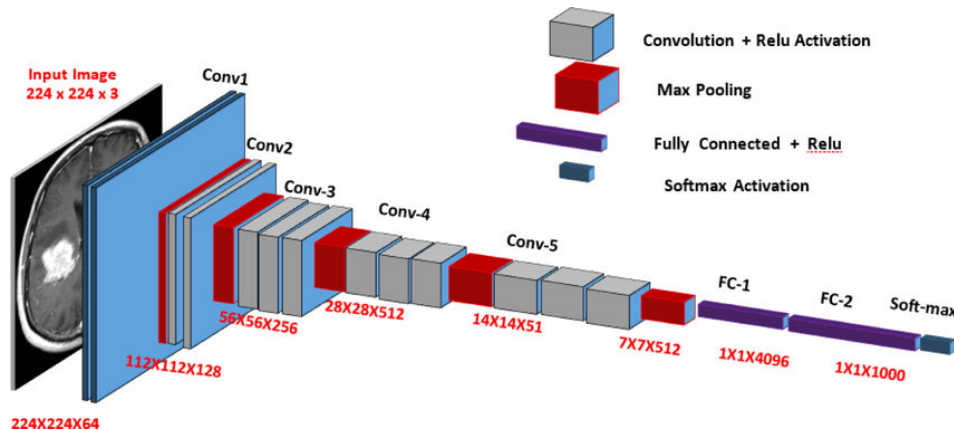


Figura 3.2. Architettura VGG16.

InceptionV3

InceptionV3, con 48 livelli, è stata sviluppata come un'estensione di GoogleNet per gestire classificazione e rilevamento immagini con elevata profondità senza aumentare eccessivamente i parametri (pesi, bias), contenendo il numero totale a circa 25 milioni di parametri (rispetto ai 60 milioni di AlexNet). Questa architettura può elaborare immagini a una risoluzione di 299×299 .

ResNet50

ResNet (Residual Network) è una rete neurale convoluzionale profonda con 50 livelli, vincitrice dell'ImageNet Large Scale Visual Recognition Challenge (ILSVRC) nel 2015. La sua architettura si basa su blocchi residui, che includono due o più livelli convoluzionali, normalizzazione batch e funzioni di attivazione. Viene detto blocco residuo perché l'input viene sommato direttamente all'output di un layer più profondo creando una "connessione di salto". Ciò permette al modello di apprendere la funzione residua invece della funzione completa. ResNet è progettata per gestire immagini con una risoluzione di 224×224 .

Questi modelli pre-addestrati rappresentano solide basi per compiti di classificazione e segmentazione in ambiti nuovi, sfruttando le loro architetture e parametri ottimizzati per estrarre caratteristiche significative dalle immagini.

Model name	Year	No. of Layers	Input Image Size
VGG16	2014	16	224*224
InceptionV3	2015	48	299*299
ResNet50	2015	50	224*224

Figura 3.3. Tabella di confronto.

3.2.5 Fine-tuning e Architettura del Modello

Al modello pre-addestrato va applicato il *fine-tuning*, una tecnica di *transfer learning* che permette di adattare un modello pre-addestrato a un nuovo compito usando un dataset più piccolo e specifico. Questa tecnica consiste nell'aggiornare alcuni parametri del modello per migliorare la sua performance sul nuovo compito, risparmiando tempo e risorse computazionali rispetto a un addestramento da zero. Il *fine-tuning* è particolarmente efficace quando il nuovo compito è simile a quello su cui il modello è stato inizialmente addestrato.

Per il fine-tuning, sono stati implementati diversi strati e tecniche aggiuntive:

- **Dropout:** Una tecnica di regolarizzazione che previene l'*overfitting* disattivando casualmente una porzione dei neuroni durante l'addestramento, simulando il training di diverse CNN in parallelo.
- **Flatten:** Uno strato che converte i tensori multidimensionali in un vettore singolo, preparando i dati per i livelli pienamente connessi.
- **Dense:** Noto anche come strato completamente connesso, collega ogni neurone di un livello a ogni neurone del livello successivo. È essenziale per estrarre caratteristiche complesse dai dati, rappresentando interazioni intricate tra input e output.

In particolare, nel codice del progetto, le immagini iniziali con dimensioni (224, 224, 3) vengono appiattite tramite `flatten()`, trasformandole in un array unidimensionale di dimensione $224 \times 224 \times 3 = 150528$, poiché i layer densi richiedono un input unidimensionale. Successivamente, sono stati aggiunti tre layer densi con 256, 128 e 64 neuroni rispettivamente, utilizzando la funzione di attivazione ReLU. Ogni layer è seguito da un'operazione di normalizzazione (`BatchNormalization()`), stabilizzando e accelerando l'addestramento, normalizzando l'output di ogni layer. Un livello di dropout è impostato a 0.4, quindi il 40% dei neuroni viene ignorato casualmente durante l'addestramento per ridurre l'overfitting.

Infine, l'ultimo layer denso ha 2 neuroni, che corrispondono alle due classi del problema di classificazione: `no_tumor` e `medulloblastoma`. La funzione di attivazione softmax è usata per generare una distribuzione di probabilità tra le classi, interpretando così le uscite del modello come probabilità di appartenenza a ciascuna classe.

3.2.6 Compilazione del modello

Il modello viene compilato utilizzando l'ottimizzatore Adam con un tasso di apprendimento pari a 1×10^{-3} . La funzione di perdita adottata è la *categorical_crossentropy*, che è una forma di funzione di perdita cross-entropy progettata per gestire classificazioni multiclasse, mentre la metrica di valutazione scelta è l'accuratezza (*accuracy*), che misura la percentuale di previsioni corrette effettuate dal modello.

3.2.6.1 Ottimizzatore Adam

Adam, che sta per *Adaptive Moment Estimation*, è un algoritmo di ottimizzazione con tasso di apprendimento adattivo, sviluppato per migliorare la velocità di addestramento nelle reti neurali profonde e raggiungere rapidamente la convergenza. L'algoritmo regola il parametro di apprendimento (learning rate) basandosi sulla cronologia del gradiente, consentendo alla rete di apprendere in modo efficiente.

Adam si basa sui principi della discesa del gradiente standard, che segue la seguente equazione:

$$\theta_{t+1} = \theta_t - \alpha g_t$$

dove θ rappresenta i parametri del modello, α il tasso di apprendimento, e g_t il gradiente della funzione di costo rispetto ai parametri.

Nella discesa del gradiente tradizionale, il tasso di apprendimento α è fisso, richiedendo un valore iniziale alto da ridurre manualmente in seguito. Tuttavia, un α troppo basso porterebbe a una convergenza lenta, mentre un valore troppo alto potrebbe causare il superamento del minimo globale.

Adam risolve questo problema in fase di addestramento del modello adattando α per ogni parametro θ , migliorando la velocità di convergenza rispetto alla discesa del gradiente con un tasso fisso.

3.2.7 Addestramento del modello

Il modello viene poi addestrato utilizzando generatori dei dati per il training e la validazione, con un numero di epoche pari a 20, poiché dai test è emerso che il modello non migliorava ulteriormente oltre tale soglia. Vengono applicati i pesi di classe definiti in precedenza, e si utilizza una callback `ReduceLROnPlateau` per ridurre il tasso di apprendimento globale nel caso in cui la perdita di validazione (`val_loss`) non mostri miglioramenti per due epoche consecutive. Il tasso di apprendimento viene quindi ridotto a un quinto del valore attuale (fattore di 0.2) e non scende al di sotto di 1×10^{-6} . In sostanza, a differenza dell'ottimizzatore Adam, che adatta il tasso di apprendimento per ciascun peso del modello in base alle sue dinamiche interne, la callback `ReduceLROnPlateau` modifica il tasso di apprendimento globale, applicandolo uniformemente a tutti i pesi. Sebbene abbiano ruoli distinti, Adam e `ReduceLROnPlateau` possono essere utilizzati insieme per ottimizzare il processo di addestramento. Anche nel caso in cui la perdita di validazione (`val_loss`) continui a migliorare e la callback non si attivi, Adam proseguirà con il suo lavoro di adattamento fine dei pesi.

3.2.8 Previsioni del modello sul validation set

Infine si calcolano le previsioni del modello, ovvero si genera la probabilità di ciascuna classe per ogni immagine nel validation set. Le probabilità vengono trasformate in etichette di classe selezionando, per ciascuna immagine, la classe con

la probabilità più alta. Questo processo crea un array di etichette predette per il set di validazione. Successivamente, le etichette predette vengono confrontate con le etichette reali, misurando la percentuale di previsioni corrette.

3.3 Segmentazione

La segmentazione delle immagini è un processo che suddivide un'immagine in "segmenti" a livello di pixel, fornendo informazioni dettagliate sulla forma e sui contorni degli oggetti presenti. A differenza della classificazione, che si concentra sull'identificazione generica, la segmentazione consente di classificare ogni pixel, associandolo a una classe specifica come "tumore" o "sfondo" nelle immagini mediche.

I principali passaggi del processo di segmentazione sono descritti di seguito in questa sezione.

3.3.1 Data Augmentation e Pre-processamento

Analogamente alla fase di classificazione, anche per la segmentazione è stato applicato un processo di data augmentation al dataset, utilizzando tecniche simili a quelle adottate per la classificazione. In fase di pre-processamento, i dati sono stati normalizzati, ridimensionando i valori delle feature delle immagini tramite divisione per 255.

3.3.2 Caricamento dei dati

Il dataset per la segmentazione è strutturato in modo simile a quello utilizzato per la classificazione, ma senza immagini della classe *no_tumore*. È presente una cartella contenente le maschere delle immagini di medulloblastoma (come descritto in precedenza in questo capitolo).

Le immagini del tumore e le rispettive maschere vengono ridimensionate a 256x256 pixel e caricate in tensori. Successivamente vengono suddivise in due insiemi distinti: un set di training, contenente l'80% delle immagini, e un set di validazione, con il restante 20%. La suddivisione tra set di training e validazione è mantenuta costante a ogni esecuzione del codice.

Inoltre, è stato effettuato un *reshape* per aggiungere una dimensione per il canale ai tensori, trasformandoli dalle dimensioni (batch_size, height, width) a (batch_size, height, width, 1). Questo canale aggiuntivo consente al modello di interpretare le immagini come in scala di grigi, rappresentate con un unico canale.

3.3.3 U-Net: Architettura e Vantaggi

L'architettura scelta per la rete neurale convoluzionale (CNN) è la U-Net. La U-Net è stata progettata per la segmentazione di immagini biomedicali ed è particolarmente utile per segmentare tumori cerebrali, dove l'accuratezza e la localizzazione per ogni singolo pixel sono fondamentali. Si distingue per la capacità di gestire dati limitati, grazie all'uso di tecniche di *data augmentation* e alla sua struttura a "U". Per questi motivi, la U-Net è stata implementata nel progetto di segmentazione.

L'architettura U-Net è composta da due fasi o percorsi principali:

- **Percorso di Contrazione (Encoder):** simile a una rete convoluzionale per la classificazione, estrae caratteristiche dell'immagine attraverso una serie di convoluzioni e riduzioni di dimensione tramite *max pooling*, rispondendo alla domanda "cosa" è presente nell'immagine.

- **Percorso di Espansione (Decoder):** ripristina la risoluzione dell'immagine alle dimensioni originali, generando una maschera segmentata e rispondendo alla domanda "dove" si trova l'oggetto. In questo percorso sono presenti anche le connessioni di salto, che collegano i livelli corrispondenti di encoder e decoder, permettendo di trasferire dettagli e contesti dalle caratteristiche apprese nel percorso di contrazione, utili per ottenere una segmentazione più precisa.

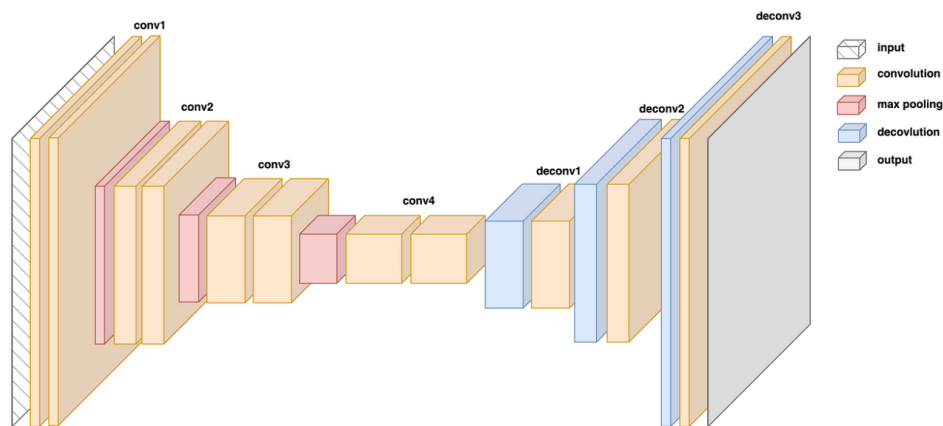


Figura 3.4. Architettura del modello U-Net. Fonte: <https://towardsdatascience.com/u-net-explained-understanding-its-image-segmentation-architecture-56e4842e313a>.

In dettaglio, il percorso di contrazione (o *downsampling*) iniziale segue l'architettura tipica di una rete convoluzionale applicando per quattro volte consecutive due convoluzioni 3x3 (con *padding* per mantenere la dimensione originale e funzione di attivazione ReLu) e un'operazione di max pooling 2x2 che ne riduce la risoluzione in termini di pixel. A ogni passaggio di downsampling, il numero di canali di caratteristiche viene raddoppiato, partendo da 64 canali e arrivando fino a 1024, mentre le dimensioni dell'immagine in input dimezzano di volta in volta, passando da 256x256 pixel a 16x16 pixel. Ogni canale rappresenta una diversa caratteristica estratta dalla rete: nei livelli iniziali, i canali potrebbero riconoscere caratteristiche di base, come bordi e colori; nei livelli più profondi, i canali combinano queste caratteristiche per identificare strutture complesse come forme o regioni specifiche dell'immagine.

Dopo la fase di contrazione e prima della fase di espansione, c'è un collo di bottiglia che consiste in due convoluzioni 3x3, senza l'applicazione di max pooling.

Da questo punto inizia il percorso di espansione (o *upsampling*), composto da quattro passaggi. In ogni passaggio, viene inizialmente eseguita una concatenazione con la mappa di caratteristiche corrispondente del percorso di contrazione (connessione di salto), seguita da un'operazione di *upsampling* (upConv 2x2), che raddoppia la risoluzione spaziale. Successivamente, vengono applicate due convoluzioni 3x3 con padding e funzione di attivazione ReLU. Questo processo aiuta a preservare le informazioni dettagliate grazie alla concatenazione con i layer del percorso di contrazione.

Nell'ultimo strato, una convoluzione 1x1 viene utilizzata per mappare ciascun vettore di caratteristiche a una singola classe con attivazione *sigmoid*, producendo così una mappa di segmentazione binaria. La rete ha in totale 19 strati convoluzionali.

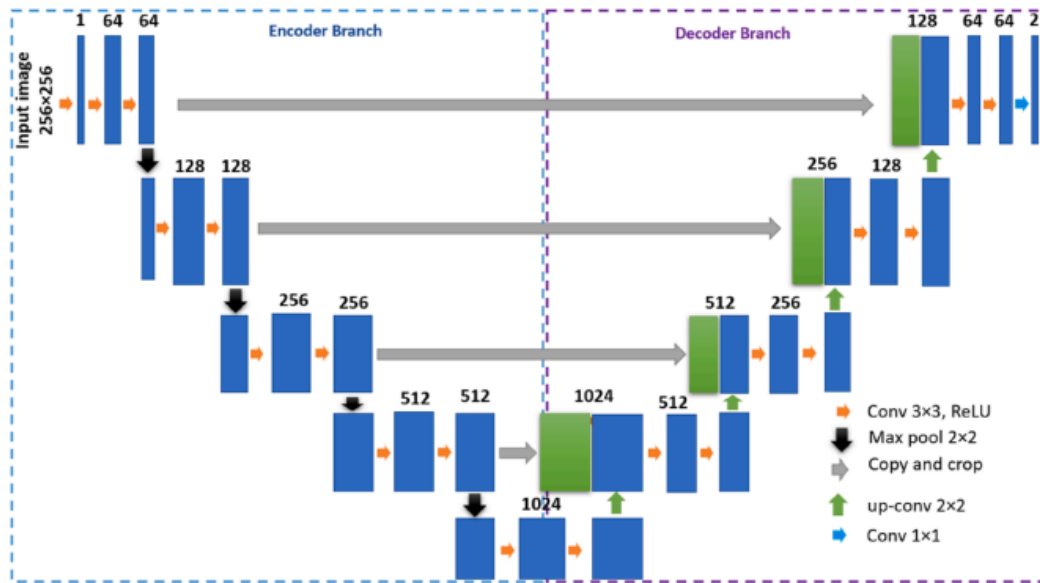


Figura 3.5. Architettura U-Net. Ogni riquadro blu rappresenta una mappa di caratteristiche multi-canale, con il numero di canali indicato sopra il riquadro. I riquadri verdi rappresentano mappe di caratteristiche copiate. Le frecce indicano le diverse operazioni.

3.3.4 Training e valutazione finale del modello

Il *batch size*, che indica il numero di immagini processate per volta nel modello, è stato impostato a 8, un valore inferiore rispetto a quello scelto per la classificazione. La segmentazione è un compito più complesso e richiede maggior tempo computazionale, poiché il modello deve identificare e delimitare accuratamente le regioni del tumore. Inoltre, un batch size ridotto consente aggiornamenti più frequenti dei pesi del modello, migliorando le prestazioni e l'accuratezza. Un batch di dimensioni minori aiuta anche a evitare il rischio di overfitting, favorendo una migliore generalizzazione.

Capitolo 4

Conclusioni

Una volta effettuate le varie scelte riguardo all'architettura del modello, l'ultima variabile da prendere in considerazione è quella relativa al numero di epoche. In questo capitolo vengono illustrati i risultati ottenuti, tenendo presente di aver utilizzato la miglior architettura, tra quelle provate durante gli esperimenti, per questo task.

4.1 Risultati della Classificazione

Nella seguente figura è possibile analizzare la matrice di confusione, che visualizza le previsioni del modello per ciascuna etichetta reale. In questo caso, 30 immagini sono state utilizzate per la validazione del tumore e 30 per la validazione del non tumore. Come si può osservare, anche con un dataset ridotto, il modello ha raggiunto un buon livello di accuratezza nella classificazione, assegnando correttamente le etichette a tutte e 60 le immagini. Tuttavia, in media, il modello commette circa due errori di classificazione per esecuzione.

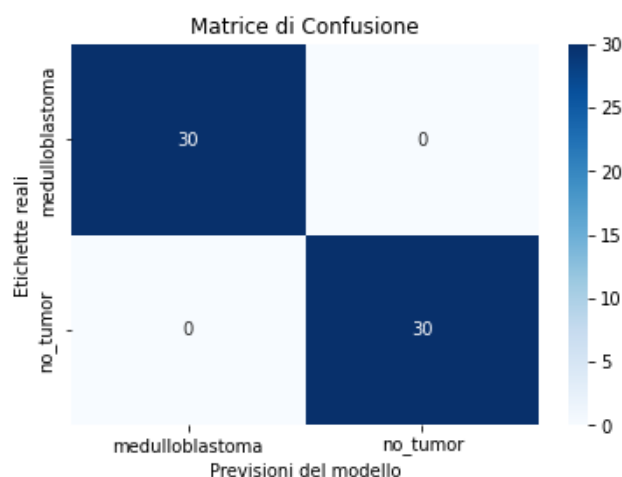


Figura 4.1. Matrice di confusione, test con 20 epoche.

È inoltre importante monitorare il valore di *Accuracy*, che misura la percentuale di previsioni corrette effettuate dal modello, e il valore di *Loss*, che indica di quanto le previsioni si discostano dai valori reali, e analizzarne l'andamento durante la fase di addestramento. Di seguito sono riportati alcuni grafici, confrontando i risultati ottenuti con un numero diverso di epoche.

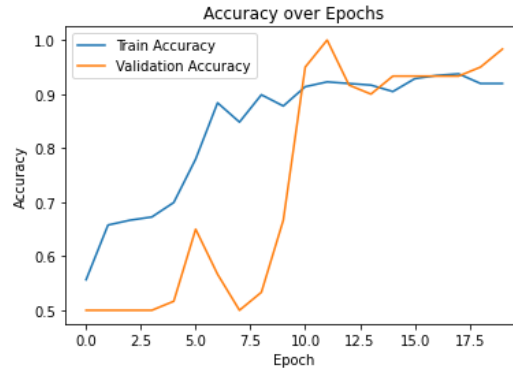


Figura 4.2. Grafico Accuracy, test con 20 epoche.

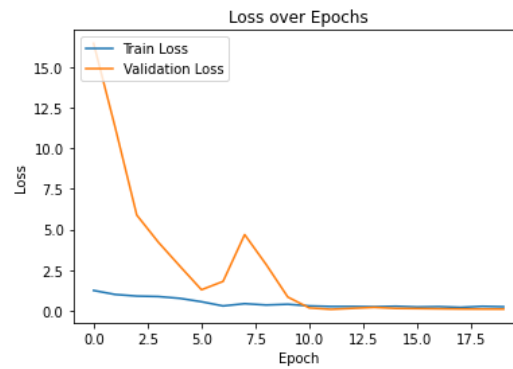


Figura 4.3. Grafico Loss, test con 20 epoche.

In sintesi, i risultati finali di *Accuracy* e *Loss* per la fase di *Train* e la fase di *Validation* ed il valore *Test Accuracy* sono riportati di seguito:

Metric	Value
Train Loss	0.321598082780838
Validation Loss	0.1266394406557083
Train Accuracy	0.9107142686843872
Validation Accuracy	0.9666666388511658
Test Accuracy	0.9666666666666667

Figura 4.4. Test con 20 epoche.

In media, i valori di Train Accuracy, Validation Accuracy e Test Accuracy sono rispettivamente 0.90, 0.94 e 0.96, calcolati sulla base di 7 esecuzioni del modello.

Per avere un quadro completo dei risultati del modello, è riportato anche il grafico dei valori di Precision, Recall e F1-Score per ogni classe. In particolare, i precedenti valori indicano:

- **Precision:** indica la proporzione di previsioni positive corrette rispetto a tutte le previsioni positive effettuate.

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

- **Recall:** indica la proporzione di casi positivi correttamente identificati rispetto al totale dei casi positivi reali.

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

- **F1-score:** è la media armonica di precision e recall, che combina entrambi gli indicatori in un unico valore.

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

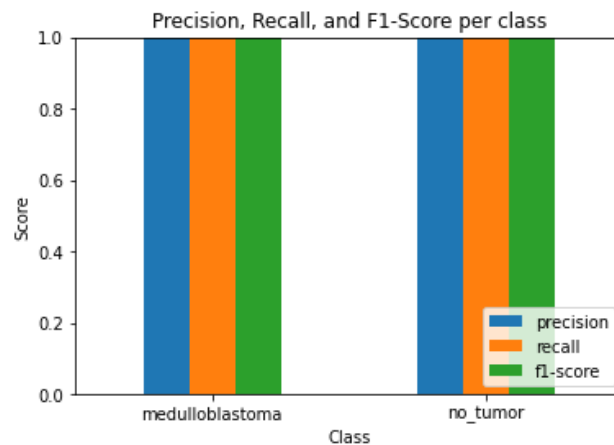


Figura 4.5. Test con 20 epoche.

Sulla base di 7 esecuzioni del modello, la media per Precision, Recall ed F1-Score è approssimativamente 0.96.

4.1.1 Classificazione con aumento del numero di epoche

Come detto in precedenza, se il numero di epoche è troppo alto, ad un certo punto il modello smette di migliorare e potrebbe adattarsi troppo ai dati utilizzati per la sua costruzione (overfitting), non riuscendo più a generalizzare su dati nuovi. Come si può osservare, in questo caso l'aumento delle epoche ha portato a un lieve peggioramento delle prestazioni. Di seguito sono riportati i risultati a dimostrazione di questa ipotesi.

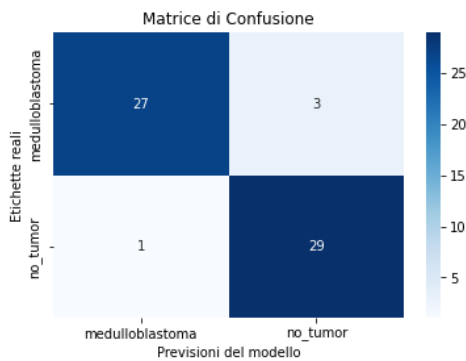


Figura 4.6. Matrice di confusione, test con 40 epoche.

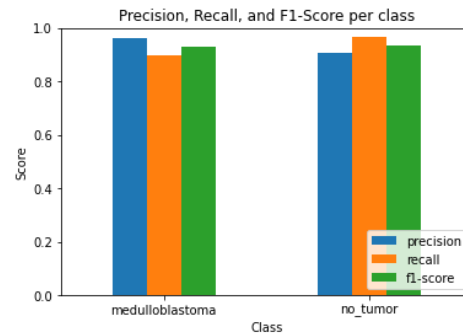


Figura 4.7. Preciso, Recall ed F1-score, test con 40 epoche.

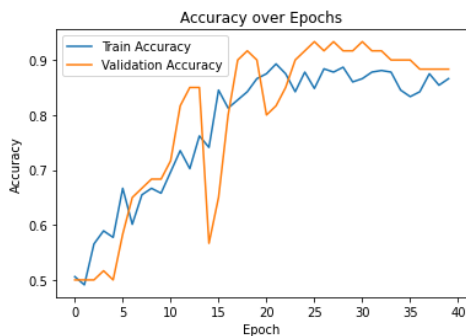


Figura 4.8. Accuratezza con 40 epoche.

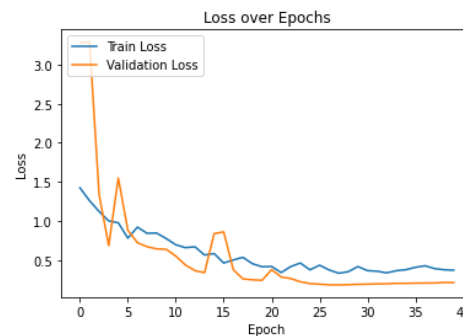


Figura 4.9. Loss con 40 epoche.

Metric	Value
Train Loss	0.36810439825057983
Validation Loss	0.21004840731620789
Train Accuracy	0.8660714030265808
Validation Accuracy	0.8833333253860474

Figura 4.10. Tabella riassuntiva, test con 40 epoche.

4.2 Risultati della Segmentazione

Nel modello di segmentazione, l'Accuracy indica la percentuale di pixel correttamente classificati (sia positivi che negativi) rispetto al totale dei pixel. Proprio per questo motivo, il valore dell'Accuracy può risultare poco attendibile. In particolare, nella maggior parte dei casi, il tumore occupa una piccola porzione dei pixel nell'immagine, mentre la restante parte appartiene allo sfondo. Di conseguenza, classificando erroneamente tutti i pixel come sfondo, si potrebbe comunque ottenere un'Accuracy elevata. Per questo motivo, nella segmentazione, le valutazioni vengono effettuate principalmente utilizzando metriche come il Dice Coefficient.

Il **coefficiente di Dice** (o **Indice di Dice**) è una misura di somiglianza tra due insiemi (in questo caso l'insieme di pixel delle maschere), ed è utilizzato principalmente per valutare la sovrapposizione tra due maschere in segmentazione delle immagini. La formula del coefficiente di Dice è la seguente:

$$\text{Dice Coefficient} = \frac{2 \cdot |A \cap B|}{|A| + |B|}$$

Dove:

- A è l'insieme di pixel che rappresentano la maschera originale,
- B è l'insieme di pixel che rappresentano la maschera predetta dal modello,
- $|A|$ e $|B|$ sono la cardinalità degli insiemi, ovvero il numero di pixel in ciascun insieme,
- $|A \cap B|$ è la cardinalità dell'intersezione tra i due insiemi, ovvero il numero di pixel comuni ad entrambi gli insiemi.

Il coefficiente di Dice è un valore compreso tra 0 e 1: un Dice pari a 1 indica una perfetta sovrapposizione tra le due maschere, cioè il modello ha segmentato esattamente l'area target; un valore di 0 indica nessuna sovrapposizione, cioè il modello non ha trovato alcuna corrispondenza con l'area target.

Di seguito sono riportati i valori del coefficiente Dice per epoca relativi al modello di segmentazione sviluppato.

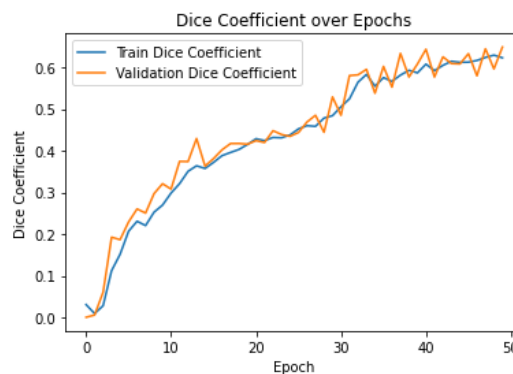


Figura 4.11. Dice Coefficient per epoca, test con 50 epoche.

Il valore di Loss da tenere in considerazione durante la fase di addestramento è il Dice Loss. Il *Dice Loss* è il complemento del *Dice Coefficient*, calcolato come:

$$\text{Dice Loss} = 1 - \text{Dice Coefficient}.$$

Esso misura la differenza tra la segmentazione prevista dal modello e quella reale, penalizzando maggiormente le sovrapposizioni errate.

Il risultato finale mostra un Dice Coefficient pari a circa 0,63. L'elevato valore di Accuracy (circa 0.98), in contrasto con un coefficiente di Dice che potrebbe essere più alto, indica che il modello soffre di overfitting e non riesce a generalizzare adeguatamente, probabilmente a causa delle dimensioni limitate del dataset.

```
Valutazione Finale - Loss: 0.030440611764788628, Accuracy:  
0.9863026738166809, Dice Coefficient: 0.6303476095199585
```

Figura 4.12. Valori output, test 50 epoche.

Su 3 esecuzioni del modello di segmentazione, il Coefficient Dice ha una media di 0.61, indicando una performance moderata nella sovrapposizione tra le maschere predette e quelle reali.

Nel risultato finale della segmentazione infatti, come si può notare dalle seguenti immagini, il modello individua correttamente l'area del tumore nell'MRI, ma non riesce a delinearne con precisione i bordi, tendendo a sfumarli.

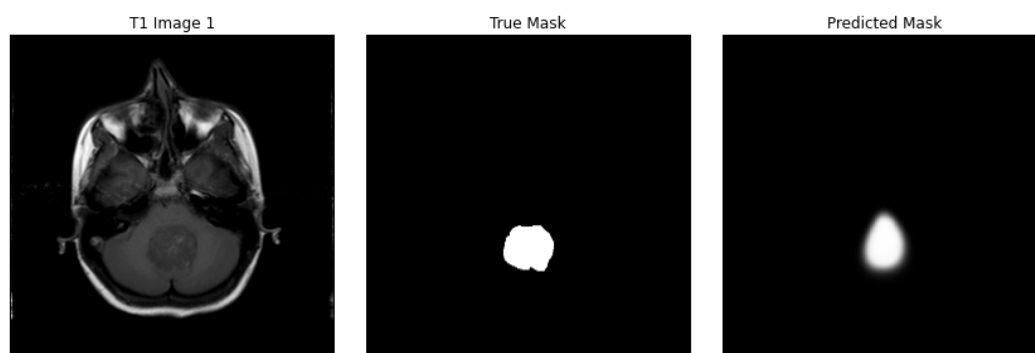


Figura 4.13. Maschera predetta, immagine MRI T1, test con 50 epoche.

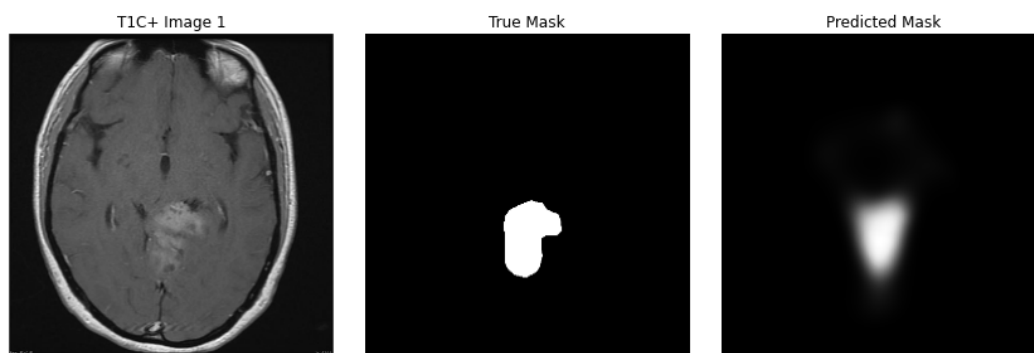


Figura 4.14. Maschera predetta, immagine MRI T1C+, test con 50 epoche.

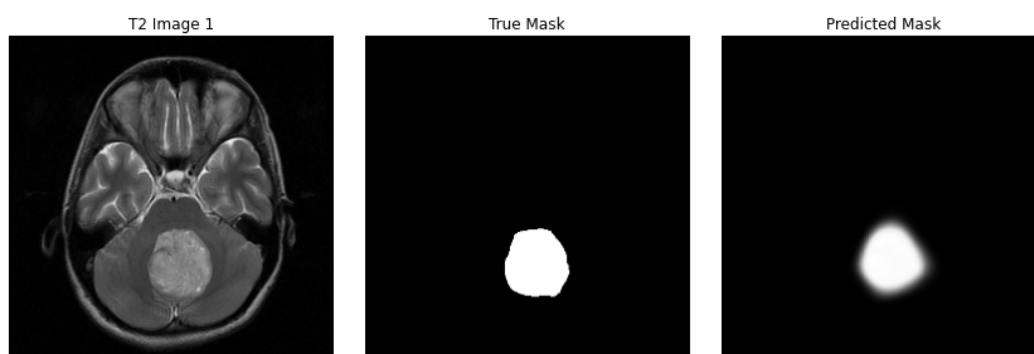


Figura 4.15. Maschera predetta, immagine MRI T2, test con 50 epoche.

4.3 Epilogo

Con il modello di classificazione si è arrivato ad ottenere ottimi risultati, nonostante il numero ridotto di immagini.

Per quanto riguarda il lavoro di segmentazione, i risultati sono fortemente influenzati dal fatto che il dataset contiene solo 111 immagini con le rispettive maschere. Inoltre, essendo presenti 3 tipi di MRI (T1, T1C+ e T2), le immagini sono state organizzate in "sottoclassi". Ad esempio, il modello per le MRI T1 ha a disposizione solo 20 immagini per l'addestramento (di cui il 20% viene utilizzato per la fase di testing). Di conseguenza, il modello non ha abbastanza risorse per segmentare correttamente i bordi, in particolare nelle immagini di tumori con forme molto irregolari o tratti discontinui.

Va anche considerato che alcune maschere potrebbero essere state create in modo errato, e quindi il modello potrebbe imparare in modo scorretto.

La tecnica del Data Augmentation ha aiutato il modello a portare il Dice Coefficient a circa 0.6, ma per migliorare ulteriormente le prestazioni sarebbe necessario disporre di un dataset più ampio e variegato, includendo immagini provenienti da diverse fonti e condizioni. Collaborazioni con ospedali potrebbero facilitare l'acquisizione di tali dati, rispettando la normativa sulla privacy e ottenendo il consenso informato dei pazienti. Questo contribuirebbe a migliorare l'accuratezza dei modelli e, in generale, a promuovere il progresso della ricerca scientifica.

Bibliografia

- [1] Millard, N. E., & De Braganca, K. C. (2016). Medulloblastoma. *Journal of Child Neurology*, 31(12), 1341-1353. doi:10.1177/0883073815600866.
- [2] Emc Elettronica. (2023). AI, Machine Learning, Deep Learning: quali le differenze? Recuperato da <https://it.emcelettronica.com/ai-machine-learning-deep-learning-quali-le-differenze>.
- [3] Abdi, A., Pouyanfar, J., Sadiq, T., Chen, S., & Iyengar, S. S. (2019). Convolutional neural networks for seizure detection. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 4(2), 127-143. Disponibile: <https://ieeexplore.ieee.org/document/8697857>.
- [4] Zhang, H., & Qie, Y. (2023). Applying Deep Learning to Medical Imaging: A Review. *Applied Sciences*, 13, 10521. <https://doi.org/10.3390/app131810521>.
- [5] Zhou, L., Ji, Q., Peng, H., Chen, F., Zheng, Y., Jiao, Z., Gong, J., & Li, W. (2023). Automatic image segmentation and online survival prediction model of medulloblastoma based on machine learning. *European Radiology*, 6. <https://link.springer.com/article/10.1007/s00330-023-10316-9>.
- [6] Hao, J., & Ho, T. K. (2019). Machine Learning Made Easy: A Review of Scikit-learn Package in Python Programming Language. *Journal of Educational and Behavioral Statistics*, 44(3), 348-361. <https://doi.org/10.3102/1076998619832248>
- [7] Zollanvari, A. (2023). *Deep Learning with Keras-TensorFlow*, in: *Machine Learning with Python*. Springer, Cham. doi:10.1007/978-3-031-33342-2_13.
- [8] Comparative Analysis of Data Visualization Libraries Matplotlib and Seaborn in Python (2021). Retrieved from <https://www.warse.org/IJATCSE/static/pdf/file/ijatcse391012021.pdf>
- [9] Masiero, G. (2020). *Deep Learning: Reti Neurali e Applicazioni*. Università degli Studi di Padova. Disponibile su: https://thesis.unipd.it/retrieve/02e8c7e8-6375-4835-a555-3b8d0c36168b/Masiero_Giacomo.pdf
- [10] Turing. (2021). *Explanation of Deep Neural Network: Multilayer Perceptron and Deep Q Network*. Disponibile su: <https://www.turing.com/kb/explanation-of-deep-neural-network-multilayer-perceptron-deep-q-network>

- [11] Deepak, R. (2021). *Multilayer Perceptron Explained with a Real-Life Example and Python Code*. Towards Data Science. Disponibile su: <https://towardsdatascience.com/multilayer-perceptron-explained-with-a-real-life-example-and-python-code-sentiment-analysis-cb408ee93141>
- [12] V7 Labs. "Understanding Cross-Entropy Loss." *V7 Labs Blog*. Available at: <https://www.v7labs.com/blog/cross-entropy-loss-guide#h2>.
- [13] Nafiz Shahriar, *What is a Convolutional Neural Network (CNN) in Deep Learning?*, 2020. Disponibile su: <https://www.linkedin.com/pulse/what-convolutional-neural-network-cnn-deep-learning-nafiz-shahriar>.
- [14] "The Importance of Data Normalization in Machine Learning," *Machine Learning Models*, 2023. [Online]. Available: <https://machinelearningmodels.org/the-importance-of-data-normalization-in-machine-learning/>.
- [15] Built In. (2023). *Adam Optimization: What it is and how it works*. Retrieved from <https://builtin.com/machine-learning/adam-optimization>.
- [16] "Detection of brain abnormality by a novel Lu-Net deep neural CNN model from MR images," *Scientific Figure on ResearchGate*, available from: https://www.researchgate.net/figure/GG-16-CNN-model-architecture-layer-wise_fig8_346259768.
- [17] M. Gupta, S. K. Sharma, and G. C. Sampada, "Classification of Brain Tumor Images Using CNN," *Exploration of Human Cognition using Artificial Intelligence in Healthcare*, Academic Editor: Z. Uddin, first published 12 October 2023. Available: <https://doi.org/10.1155/2023/2002855>.
- [18] Nascimento, A. C. and Silva, F. R. and Lima, L. A. and others, "Brain Tumor Classification using VGG 16, ResNet50, and Inception V3 Transfer Learning Models" *2024 6th International Conference on Data Science and Machine Learning Applications (ICDMA)*, 2024. Available: <https://ieeexplore.ieee.org/document/10101252>
- [19] Analytics Vidhya. (2022). Image Segmentation with U-Net. Retrieved from <https://www.analyticsvidhya.com/blog/2022/10/image-segmentation-with-u-net/#h-introduction>
- [20] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241. Springer, 2015. <https://arxiv.org/pdf/1505.04597>.
- [21] Ahmed M. Gab Allah, Amany M. Sarhan, and Nada M. Elshennawy. Edge U-Net: Brain tumor segmentation using MRI based on deep U-Net model with boundary information. *Expert Systems with Applications*, 213:118833, 2023. <https://www.sciencedirect.com/science/article/pii/S0957417422018516>.

